Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

ОТЧЁТ

по лабораторной работе № 1

«Генерация бинарного кода Грея. Мультимножества и операции над ними»

по дисциплине «Дискретная математика»

Выполнил
студент гр. 5130201/50302 _____ Р. Ю. Дужнов

Проверил
преподаватель _____ А. В. Востров

« ____ » _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Математическое описание 4
1.1 Множества и мультимножества 4
1.2 Операции над мультимножествами 5
1.3 Бинарный код Грея 7
1.4 Алгоритм автоматического заполнения 8
2 Особенности реализации 10
2.1 Средства разработки и структура программы 10
2.2 Допущения и входные данные 11
2.3 Код Грея (`GrayCode.hpp`, `GrayCode.cpp`) 12
2.4 Генератор случайных чисел (`Random.hpp`, `Random.cpp`) 17
2.5 Мультимножество (`Multiset.hpp`, `Multiset.cpp`) 19
2.6 Универсум (`Universe.hpp`, `Universe.cpp`) 30
2.7 Ввод (`Input.hpp`, `Input.cpp`) 31
2.8 Меню (`Menu.hpp`, `Menu.cpp`) 36
2.9 Главная функция (`main.cpp`) 49
2.10 Замер времени (`bench.cpp`) 50
2.11 Ограничения программы 52
2.12 Обработка ошибок и защита ввода 53
3 Результаты работы 54
3.1 Корректная работа при $n = 3$ 54
3.2 Некорректный ввод команд и параметров 57
3.3 Ошибки при ручном заполнении 58
3.4 Граничные значения мощности и кратности 60
3.5 Пустой универсум, пересоздание и конец ввода 62
3.6 Наибольшая разрядность 63
ЗАКЛЮЧЕНИЕ 67
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 69

ВВЕДЕНИЕ

В работе требуется разработать программу для работы с мультимножествами, элементами которых служат слова бинарного кода Грея.

Программа должна генерировать бинарный код Грея разрядности n, которую задаёт пользователь, и заполнять его словами универсум мультимножеств. На основе универсума программа формирует два мультимножества A и B, мощности которых задаёт пользователь, а способ заполнения каждого из них — вручную или автоматически — пользователь выбирает сам.

Над мультимножествами A и B программа должна вычислять и выводить результаты операций объединения, пересечения, разности, симметрической разности, дополнения каждого из мультимножеств до универсума, а также арифметических суммы, разности, произведения и деления. Кроме того, программа должна защищать работу от некорректного ввода и предоставлять пользователю меню.

1 Математическое описание

1.1 Множества и мультимножества

Множество — это любая определённая совокупность объектов. Объекты, из которых составлено множество, называются его элементами. Элементы множества различны и отличимы друг от друга [1]. Запись $x \in X$ означает, что x — элемент множества X.

Мультимножеством A над конечным множеством X называется функция кратности

$$k_A : X \to \{0, 1, 2, \ldots\}. \tag{1}$$

Множество X называется носителем A, значение $k_A(x)$ называется кратностью элемента x в A и равно числу вхождений x в A [1, 2]. Элемент x принадлежит A тогда и только тогда, когда $k_A(x) > 0$. В отчёте и в выводе программы мультимножество записывается в фигурных скобках перечнем пар «элемент × кратность» для элементов с ненулевой кратностью, например `{000×2, 011×1}`; если все кратности равны нулю, записывается `{}`.

Мощность мультимножества — сумма значений его функции кратности [1]:

$$|A| = \sum_{x \in X} k_A(x). \tag{2}$$

Мультимножество A — подмультимножество мультимножества B над тем же носителем, если

$$A \subseteq B \iff k_A(x) \leqslant k_B(x) \ \text{ для всех } x \in X. \tag{3}$$

Из (2) и (3): если $A \subseteq B$, то $|A| \leqslant |B|$.

Универсумом называется мультимножество U над носителем X, для которого $A \subseteq U$ для каждого рассматриваемого в задаче мультимножества A над X. По (3) это означает, что $0 \leqslant k_A(x) \leqslant k_U(x)$ для всех $x \in X$, то есть $k_U(x)$ — наибольшая кратность, с которой элемент x может входить в мультимножества задачи. Отсюда $0 \leqslant |A| \leqslant |U|$.

В программе носитель X — множество слов бинарного кода Грея разрядности n (подраздел 1.3); при $n \geqslant 1$ число слов $N = 2^n$. Универсум U — мультимножество над X, функция кратности которого на всех словах равна одному и тому же числу k, $1 \leqslant k \leqslant 100$. Число k вводит пользователь; если

вместо k введена пустая строка, программа выбирает k случайно от 1 до 100:

$$k_U(x) = k \quad \text{для всех } x \in X, \qquad |U| = k \cdot 2^n, \qquad n \geqslant 1. \tag{4}$$

Мультимножества A и B строятся как подмультимножества U, поэтому по (3) $k_A(x) \leqslant k$ и $k_B(x) \leqslant k$ для всех $x \in X$, а по (2) $0 \leqslant |A|, |B| \leqslant |U|$. При $n = 0$ алгоритм из [1] выдаёт один код — код пустого подмножества ($2^0 = 1$); в программе принято, что код Грея при $n = 0$ пуст: $N = 0$, $|U| = 0$.

1.2 Операции над мультимножествами

Для подмножеств A и B множества X, которое здесь играет роль универсального множества, операции определяются так [1]:

$$A \cup B = \{x \mid x \in A \vee x \in B\}, \qquad A \cap B = \{x \mid x \in A \;\&\; x \in B\},$$
$$A \setminus B = \{x \mid x \in A \;\&\; x \notin B\}, \qquad A \triangle B = (A \cup B) \setminus (A \cap B), \qquad \overline{A} = X \setminus A.$$

Результат каждой операции над мультимножествами $A \subseteq U$ и $B \subseteq U$ является мультимножеством над тем же носителем X, поэтому по определению (1) он задаётся своей функцией кратности. Она обозначается индексом с обозначением операции: для $C = A \cup B$ это $k_C = k_{A \cup B} \colon X \to \{0, 1, 2, \ldots\}$. В программе значение функции кратности результата на элементе x вычисляется по кратностям этого элемента в A, B и U. Объединение, пересечение и дополнения:

$$k_{A \cup B}(x) = \max\bigl(k_A(x),\ k_B(x)\bigr), \tag{5}$$
$$k_{A \cap B}(x) = \min\bigl(k_A(x),\ k_B(x)\bigr), \tag{6}$$
$$k_{\overline{A}}(x) = k_U(x) - k_A(x), \tag{7}$$
$$k_{\overline{B}}(x) = k_U(x) - k_B(x). \tag{8}$$

Кратности (7) и (8) неотрицательны, так как $k_A(x) \leqslant k_U(x)$ и $k_B(x) \leqslant k_U(x)$.

Для множеств из определений разности и дополнения следует равенство

$$A \setminus B = A \cap \overline{B}, \tag{9}$$

так как обе части состоят из элементов X, которые принадлежат A и не при-

надлежат B. В программе разность мультимножеств вычисляется именно в форме (9). Подставляя в (9) формулы пересечения (6) и дополнения (8), получаем

$$k_{A\setminus B}(x) = \min(k_A(x),\ k_{\overline{B}}(x)) = \min(k_A(x),\ k_U(x) - k_B(x)), \tag{10}$$

$$k_{B\setminus A}(x) = \min(k_B(x),\ k_{\overline{A}}(x)) = \min(k_B(x),\ k_U(x) - k_A(x)). \tag{11}$$

Для множеств из определения симметрической разности следует равенство

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \tag{12}$$

так как обе части состоят из элементов, которые принадлежат ровно одному из множеств A и B. В программе симметрическая разность мультимножеств вычисляется именно в форме (12). Подставляя в (12) формулы объединения (5) и разностей (10), (11), получаем

$$k_{A\triangle B}(x) = \max\Big(\min(k_A(x),\ k_U(x)-k_B(x)),\ \min(k_B(x),\ k_U(x)-k_A(x))\Big). \tag{13}$$

Кратность (13) в общем случае не равна $|k_A(x)-k_B(x)|$: при $k_U(x) = 3, k_A(x) = 1, k_B(x) = 2$ она равна $\max(\min(1,1),\min(2,2)) = 2$.

Арифметические операции вычисляются над значениями функций кратности:

$$k_{A+B}(x) = \min(k_A(x) + k_B(x),\ k_U(x)), \tag{14}$$

$$k_{A-B}(x) = \max(k_A(x) - k_B(x),\ 0), \tag{15}$$

$$k_{B-A}(x) = \max(k_B(x) - k_A(x),\ 0), \tag{16}$$

$$k_{A\cdot B}(x) = \min(k_A(x) \cdot k_B(x),\ k_U(x)), \tag{17}$$

$$k_{A/B}(x) = \begin{cases} k_A(x) \operatorname{div} k_B(x), & k_B(x) > 0, \\ 0, & k_B(x) = 0, \end{cases} \tag{18}$$

$$k_{B/A}(x) = \begin{cases} k_B(x) \operatorname{div} k_A(x), & k_A(x) > 0, \\ 0, & k_A(x) = 0, \end{cases} \tag{19}$$

где div — целочисленное деление. Сумма и произведение могут превысить $k_U(x)$, поэтому ограничены этой величиной сверху; разность ограничена

снизу нулём, так как по определению (1) кратность неотрицательна. Разность (10) и арифметическая разность (15) — разные операции: при $k_U(x) = 3$, $k_A(x) = 2$, $k_B(x) = 1$ имеем $k_{A\setminus B}(x) = 2$ и $k_{A-B}(x) = 1$.

Для множеств пересечение, объединение и разность подмножеств универсального множества являются его подмножествами [1]. Для мультимножеств выполняется аналогичное утверждение: если $A \subseteq U$ и $B \subseteq U$, то по (3) результаты всех операций (5)–(19) являются подмультимножествами U. Максимум и минимум чисел, не превосходящих $k_U(x)$, не превосходят $k_U(x)$; $0 \leqslant k_U(x) - k_A(x) \leqslant k_U(x)$; обе разности не больше уменьшаемого: $\min(k_A(x), \ldots) \leqslant k_A(x)$ и $\max(k_A(x) - k_B(x), 0) \leqslant k_A(x)$; сумма и произведение ограничены величиной $k_U(x)$ по определению; при $k_B(x) \geqslant 1$ частное $k_A(x) \operatorname{div} k_B(x) \leqslant k_A(x)$, а при $k_B(x) = 0$ оно равно нулю. Для операций $B \setminus A$, $B - A$ и B/A рассуждение то же с перестановкой A и B.

1.3 Бинарный код Грея

Бинарным кодом Грея разрядности n называется последовательность всех 2^n двоичных кодов длины n, в которой любые два соседних кода различаются ровно в одном разряде [1].

Код Грея строится алгоритмом из [1] (алгоритмы 1.2 и 1.3). Начальный код состоит из нулей. На шаге $i = 1, 2, \ldots, 2^n - 1$ меняется разряд с номером $Q(i)$, где $Q(i)$ — количество нулей на конце двоичной записи числа i, увеличенное на 1; в примере выполнения алгоритма в [1] разряд с номером 1 — крайний правый. Корректность алгоритма доказана в [1] индукцией по n.

В программе разряды нумеруются справа, начиная с 0, поэтому номер разряда, который меняется на шаге i, равен

$$p(i) = Q(i) - 1 = \text{количество нулей на конце двоичной записи числа } i. \quad (20)$$

При $n \geqslant 1$, если обозначить i-й код последовательности через g_i,

$$g_0 = 0, \qquad g_i = g_{i-1} \oplus 2^{p(i)}, \quad i = 1, 2, \ldots, 2^n - 1, \qquad (21)$$

где $\oplus$ — побитовое исключающее ИЛИ.

На вход алгоритма подаётся разрядность n, $0 \leqslant n \leqslant 19$; на выходе получается последовательность слов кода Грея $g_0, g_1, \ldots, g_{N-1}$, которая образует

носитель X. Код строит функция `generate_gray_code` (п. 2.3.4), номер разряда $p(i)$ вычисляет функция `changed_bit` (п. 2.3.3). Алгоритм в программе выполняется так:

1) если $n = 0$, возвращается пустая последовательность, и алгоритм завершается;
2) если $n \geqslant 1$, в последовательность записывается код $g_0 = 0$;
3) для $i = 1, 2, \ldots, 2^n - 1$: число i делится на 2, пока оно чётно, число делений равно $p(i)$; в текущем коде инвертируется разряд $p(i)$, и полученный код g_i дописывается в последовательность;
4) последовательность возвращается.

Чтобы найти $p(i)$, число i проверяется на чётность $p(i)+1$ раз. Проверка с номером $t+1$, где $t = 0, 1, \ldots, n-1$, выполняется только для тех i, которые делятся на 2^t; среди чисел от 1 до $2^n - 1$ таких чисел $2^{n-t} - 1$. Поэтому общее число проверок

$$\sum_{i=1}^{2^n-1} (p(i) + 1) = \sum_{t=0}^{n-1} (2^{n-t} - 1) = 2^{n+1} - n - 2 < 2 \cdot 2^n. \tag{22}$$

Кроме проверок, на каждом из $2^n - 1$ шагов выполняются одно инвертирование разряда и одна запись кода. Трудоёмкость алгоритма — $O(2^n)$, память — 2^n чисел под получаемую последовательность.

1.4 Алгоритм автоматического заполнения

Автоматическое заполнение строит случайное подмультимножество универсума заданной мощности. На вход алгоритма подаются универсум U и мощность m, $0 \leqslant m \leqslant |U|$; на выходе получается мультимножество R над носителем X, для которого $R \subseteq U$ и $|R| = m$. В программе алгоритм выполняет функция `random_submultiset` (п. 2.5.21). Алгоритм:

1) создаётся мультимножество R над X с $k_R(x) = 0$ для всех $x \in X$ и список F элементов x, для которых $k_R(x) < k_U(x)$; в начале в F входят все x, у которых $k_U(x) > 0$;
2) m раз выполняется: из списка F равновероятно выбирается элемент x, его кратность $k_R(x)$ увеличивается на 1; если $k_R(x)$ стала равна $k_U(x)$, элемент x удаляется из F: на его место в списке ставится последний элемент, и список укорачивается на 1;

3) возвращается R.

Шаги нумеруются $s = 0, 1, \ldots, m - 1$. Перед шагом s в R распределено s единиц, поэтому свободных мест остаётся $|U| - s > 0$, так как $s < m \leqslant |U|$. Следовательно, список F не пуст ни на одном шаге, кратность ни одного элемента не превышает $k_U(x)$, $R \subseteq U$ и $|R| = m$.

До начала шагов выполняется $O(2^n)$ операций: вычисление $|U|$ для проверки условия $m \leqslant |U|$ (сумма 2^n кратностей), создание R с 2^n нулевыми кратностями и построение списка F. Каждый из m шагов состоит из выбора случайной позиции в F, увеличения одной кратности и, возможно, удаления элемента переносом последнего; эти действия выполняются за $O(1)$, причём выбор случайного числа — в среднем за $O(1)$. Так как $m \leqslant |U| = k \cdot 2^n$ и $k \leqslant 100$, число шагов не превосходит $100 \cdot 2^n$, поэтому трудоёмкость алгоритма — $O(2^n)$, дополнительная память — $O(2^n)$ под R и F.

2 Особенности реализации

2.1 Средства разработки и структура программы

Программа написана на языке C++ стандарта C++14. Для сборки в проекте есть файл `CMakeLists.txt` (листинг 1): он задаёт стандарт C++14 и две цели — саму программу `lab1` и программу замера времени `bench`.

Листинг 1 — Файл сборки `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.10)
project(lab1 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CORE_SOURCES
    src/core/GrayCode.cpp
    src/core/Random.cpp
    src/core/Multiset.cpp
    src/core/Universe.cpp
)

add_executable(lab1 src/main.cpp src/ui/Input.cpp src/ui/Menu.cpp ${CORE_SOURCES})
target_include_directories(lab1 PRIVATE src)

add_executable(bench tools/bench.cpp ${CORE_SOURCES})
target_include_directories(bench PRIVATE src)
```

Для отчёта программа собиралась компилятором g++ (MinGW.org GCC 6.3.0, 32-разрядная сборка) напрямую:

```
g++ -std=c++14 -O2 -Wall -Wextra -Isrc src/main.cpp
    src/ui/*.cpp src/core/*.cpp -o lab1.exe
```

Предупреждений при компиляции нет. Программа запускается без параметров либо с параметром `--seed N`: тогда генератор случайных чисел получает зерно N, и запуск с тем же вводом повторяется полностью. Без параметра зерно получает функция `make_seed`. В обоих случаях зерно выводится первой строкой.

Исходный код разделён на ядро (`src/core`), где реализованы код Грея и мультимножества, и интерфейс (`src/ui`), где реализованы ввод и меню. Ядро не выводит ничего на экран, а интерфейс не вычисляет операции сам. Назначение файлов и подразделы, в которых описаны их функции, приведены в табл. 1.

Таблица 1 — Файлы проекта

Файл	**Назначение**	**Подраздел**
`CMakeLists.txt`	сборка программы `lab1` и программы замера `bench`	2.1
`src/core/GrayCode.hpp`, `src/core/GrayCode.cpp`	константы разрядности, исключения кода Грея, генерация кода Грея, класс `CodeSet`	2.3
`src/core/Random.hpp`, `src/core/Random.cpp`	генератор `std::mt19937` и функции работы с ним	2.4
`src/core/Multiset.hpp`, `src/core/Multiset.cpp`	исключения мультимножеств, класс `Multiset`, автоматическое заполнение, вывод	2.5
`src/core/Universe.hpp`, `src/core/Universe.cpp`	константа `MAX_MULTIPLICITY`, построение равномерного универсума	2.6
`src/ui/Input.hpp`, `src/ui/Input.cpp`	чтение строк и чисел с проверкой	2.7
`src/ui/Menu.hpp`, `src/ui/Menu.cpp`	класс `Menu`: команды, заполнение мультимножеств, вывод таблицы, операций и матрицы	2.8
`src/main.cpp`	функция `main`	2.9
`tools/bench.cpp`	замер времени самых долгих действий при разных n	2.10

2.2 Допущения и входные данные

При реализации приняты следующие допущения. Универсум равномерный: все слова кода Грея входят в него с одной и той же кратностью $k_U(x) = k$, которую задаёт пользователь или выбирает программа. Разрядность ограничена условием $0 \leqslant n \leqslant 19$, кратность — условием $1 \leqslant k \leqslant 100$; выбор этих пределов обоснован в подразделе 2.11. При $n = 0$ код Грея по соглашению из подраздела 1.1 считается пустым, поэтому универсум пуст, кратность не запрашивается, а мощности A и B могут быть только нулевыми. Мультимножества A и B всегда строятся как подмультимножества универсума, поэтому их мощности лежат в пределах от 0 до $|U|$. Пересоздание универсума сбрасывает уже заданные A и B, так как они построены над прежним носителем. Если заполнение отменено, прежнее мультимножество сохраняется.

Входные данные программы, их допустимые значения и реакция программы на некорректный и пустой ввод приведены в табл. 2.

Таблица 2 — Входные данные программы

Величина	**Допустимые значения**	**Реакция на некорректный и пустой ввод**
Команда меню	одна буква из `u a b t o m h q`, регистр не важен	сообщение «не команда» или «пустая строка» и повторный запрос; если команда недоступна, выводится, чего не хватает
Разрядность n	целое от 0 до 19	повторный запрос с причиной: пустая строка, не число, число вне отрезка
Кратность $k_U(x) = k$	целое от 1 до 100; при $n = 0$ не запрашивается	пустая строка — случайное k от 1 до 100; иначе повторный запрос с причиной
Мощности $\|A\|$, $\|B\|$	целое от 0 до $\|U\|$	пустая строка — случайная мощность от 0 до $\|U\|$; иначе повторный запрос
Способ заполнения	`r` — случайно, `m` — вручную	пустая строка — отмена, мультимножество не меняется; иначе повторный запрос
Строка ручного ввода	пары `код:кратность` через пробел, код из n символов 0 и 1, кратность от 1	строка отклоняется целиком с указанием причины; пустая строка — отмена

2.3 Код Грея (`GrayCode.hpp`, `GrayCode.cpp`)

Файлы содержат константы, ограничивающие разрядность, классы исключений кода Грея, функции генерации кода Грея и перевода слова в строку, а также класс `CodeSet`, который хранит носитель X.

2.3.1 Константы `MAX_DEPTH` и `MAX_TYPE_DEPTH`

Константа `MAX_DEPTH` = 19 — наибольшая разрядность, которую можно ввести в меню; её выбор обоснован в подразделе 2.11. Константа `MAX_TYPE_DEPTH = sizeof(size_t) * 8 - 1` — наибольшая разрядность, при которой число 2^n помещается в тип `size_t`; в 32-разрядной сборке она равна 31. Объявления констант приведены в листинге 2.

Листинг 2 — Константы `MAX_DEPTH` и `MAX_TYPE_DEPTH`

```
1 const size_t MAX_DEPTH = 19;
2
3 const size_t MAX_TYPE_DEPTH = sizeof(size_t) * 8 - 1;
```

2.3.2 Классы исключений кода Грея

Класс `GrayCodeError` — наследник `std::runtime_error` и базовый класс ошибок кода Грея; его конструктор принимает текст сообщения. От

него наследуются два класса. Исключение `InvalidDepthError` с сообщением «2^n не помещается в size_t» выбрасывается при разрядности больше `MAX_TYPE_DEPTH`. Исключение `CodeIndexError` с сообщением «номер кода вне последовательности» выбрасывается при обращении к слову с номером $i \geqslant N$. Классы приведены в листинге 3.

Листинг 3 — Классы исключений кода Грея

```
class GrayCodeError : public std::runtime_error{
public:
    explicit GrayCodeError(const std::string& message):std::runtime_error(message){}
};

class InvalidDepthError : public GrayCodeError{
public:
    InvalidDepthError():GrayCodeError("2^n не помещается в size_t"){}
};

class CodeIndexError : public GrayCodeError{
public:
    CodeIndexError():GrayCodeError("номер кода вне последовательности"){}
};
```

2.3.3 Функция `changed_bit`

Функция вычисляет номер разряда $p(i)$, который инвертируется на шаге i генерации кода Грея по формуле (20). На вход подаётся номер шага i, $1 \leqslant i \leqslant 2^n - 1$; при $i = 0$ цикл не завершается, поэтому функция с таким значением не вызывается. На выходе получается $p(i)$, то есть число нулей в конце двоичной записи i. Пока текущее значение чётно, оно делится на 2; число выполненных делений возвращается. Функция объявлена как `static` и используется только в `generate_gray_code`. Текст функции приведён в листинге 4.

Листинг 4 — Функция `changed_bit`

```
static size_t changed_bit(size_t i){
    size_t q = 0;
    size_t j = i;
    while(j % 2 == 0){
        j /= 2;
        q++;
    }
    return q;
}
```

2.3.4 Функция `generate_gray_code`

Функция строит последовательность слов кода Грея по алгоритму из подраздела 1.3. На вход подаётся разрядность `depth` = n. На выходе получается вектор `std::vector<size_t>` слов $g_0, \dots, g_{N-1}$; слово хранится числом, у которого разряд с номером j (справа, с нуля) равен j-му разряду слова. При $n = 0$ вектор пуст. При `depth` > `MAX_TYPE_DEPTH` выбрасывается `InvalidDepthError`. Память под 2^n слов резервируется заранее, затем в вектор записывается $g_0 = 0$, и для $i = 1, \dots, 2^n - 1$ по формуле (21) дописывается $g_i = g_{i-1} \oplus 2^{p(i)}$, где $p(i)$ возвращает `changed_bit`. Трудоёмкость — $O(2^n)$. Текст функции приведён в листинге 5.

Листинг 5 — Функция `generate_gray_code`

```
std::vector<size_t> generate_gray_code(size_t depth){
    if(depth > MAX_TYPE_DEPTH){
        throw InvalidDepthError();
    }
    std::vector<size_t> codes;
    if(depth == 0){
        return codes;
    }
    size_t count = size_t(1) << depth;
    codes.reserve(count);
    size_t code = 0;
    codes.push_back(code);
    for(size_t i = 1; i < count; i++){
        size_t p = changed_bit(i);
        code ^= size_t(1) << p;
        codes.push_back(code);
    }
    return codes;
}
```

2.3.5 Функция `code_to_string`

Функция переводит слово кода Грея, хранящееся числом, в строку из символов 0 и 1. На вход подаются слово `code` и разрядность `depth`; на выходе получается строка длины `depth`, старший разряд слева. Разряды перебираются от `depth` – 1 до 0, и в строку дописывается символ, равный значению разряда. Трудоёмкость — $O(n)$. Текст функции приведён в листинге 6.

Листинг 6 — Функция `code_to_string`

```
std::string code_to_string(size_t code, size_t depth){
    std::string res;
    for(size_t bit = depth; bit-- > 0;){
        res += ((code >> bit) & 1) ? '1' : '0';
    }
    return res;
}
```

2.3.6 Класс `CodeSet`

Класс хранит носитель X — слова кода Грея разрядности n — и обеспечивает переход от номера элемента к слову и обратно. Поле `depth` содержит разрядность n. Поле `codes` типа `std::vector<size_t>` содержит слова $g_0, \dots, g_{N-1}$ в порядке генерации: элемент x_i носителя — слово `codes[i]`. Поле `positions` той же длины содержит обратную таблицу: `positions[c]` равно номеру слова c в `codes`. Так как код Грея содержит все 2^n слов длины n, номер определён для любого слова. Слово хранится числом и переводится в строку только при выводе. Объявление класса приведено в листинге 7.

Листинг 7 — Класс `CodeSet`

```
class CodeSet{
    size_t depth;
    std::vector<size_t> codes;
    std::vector<size_t> positions;

public:
    explicit CodeSet(size_t depth);

    size_t get_depth() const;
    size_t size() const;
    std::string get_code(size_t i) const;
    bool find(const std::string& code, size_t& index) const;
};
```

2.3.7 Конструктор `CodeSet`

Конструктор создаёт носитель разрядности `depth`, которая подаётся на вход. Поле `codes` заполняется вызовом `generate_gray_code(depth)`, затем для каждого i записывается `positions[codes[i]] = i`. При `depth` > `MAX_TYPE_DEPTH` исключение `InvalidDepthError` передаётся из `generate_gray_code`. Трудоёмкость — $O(2^n)$. Текст конструктора приведён в листинге 8.

Листинг 8 — Конструктор `CodeSet`

```
CodeSet::CodeSet(size_t
depth):depth(depth),codes(generate_gray_code(depth)),positions(codes.size()){
    for(size_t i = 0; i < codes.size(); i++){
        positions[codes[i]] = i;
    }
}
```

2.3.8 Метод `CodeSet::get_depth`

Метод не имеет входных данных и возвращает разрядность n носителя. Текст метода приведён в листинге 9.

Листинг 9 — Метод `CodeSet::get_depth`

```
size_t CodeSet::get_depth() const{
    return depth;
}
```

2.3.9 Метод `CodeSet::size`

Метод не имеет входных данных и возвращает число N элементов носителя, равное длине `codes`: 2^n при $n \geqslant 1$ и 0 при $n = 0$. Текст метода приведён в листинге 10.

Листинг 10 — Метод `CodeSet::size`

```
size_t CodeSet::size() const{
    return codes.size();
}
```

2.3.10 Метод `CodeSet::get_code`

Метод возвращает элемент x_i носителя в виде строки. На вход подаётся номер i; на выходе получается строка `code_to_string(codes[i], depth)`. При $i \geqslant N$ выбрасывается `CodeIndexError`. Текст метода приведён в листинге 11.

Листинг 11 — Метод `CodeSet::get_code`

```
std::string CodeSet::get_code(size_t i) const{
    if(i >= codes.size()){
        throw CodeIndexError();
    }
    return code_to_string(codes[i], depth);
}
```

2.3.11 Метод `CodeSet::find`

Метод находит номер слова, заданного строкой. На вход подаются строка `code` и ссылка `index`, в которую записывается результат. Если строка имеет длину n и состоит из символов 0 и 1, метод записывает в `index` номер слова и возвращает `true`; иначе, а также при пустом носителе, возвращает `false`, и `index` не меняется. Строка переводится в число c за $O(n)$, а номер берётся из `positions[c]` без просмотра `codes`. Текст метода приведён в листинге 12.

Листинг 12 — Метод `CodeSet::find`

```
bool CodeSet::find(const std::string& code, size_t& index) const{
    if(codes.empty() || code.size() != depth){
        return false;
    }
    size_t value = 0;
    for(size_t i = 0; i < code.size(); i++){
        if(code[i] != '0' && code[i] != '1') return false;
        value = value * 2 + (code[i] - '0');
    }
    index = positions[value];
    return true;
}
```

2.4 Генератор случайных чисел (`Random.hpp`, `Random.cpp`)

Файлы содержат генератор псевдослучайных чисел и функции работы с ним. Генератор используется при автоматическом заполнении и при случайном выборе кратности k и мощности мультимножества.

2.4.1 Функция `generator`

Функция не имеет входных данных и возвращает ссылку на единственный объект генератора `std::mt19937`, который хранится в статической локальной переменной и создаётся при первом вызове. Функция объявлена как

`static` и используется только в `Random.cpp`. Текст функции приведён в листинге 13.

Листинг 13 — Функция `generator`

```
static std::mt19937& generator(){
    static std::mt19937 gen;
    return gen;
}
```

2.4.2 Функция `make_seed`

Функция не имеет входных данных и возвращает зерно генератора типа `unsigned`. Зерно равно побитовому исключающему ИЛИ значения `std::random_device` и текущего значения часов `std::chrono::high_resolution_clock`. Время примешивается потому, что в MinGW `std::random_device` при каждом запуске выдаёт одну и ту же последовательность. Текст функции приведён в листинге 14.

Листинг 14 — Функция `make_seed`

```
unsigned make_seed(){
    std::random_device device;
    unsigned time = static_cast<unsigned>(
        std::chrono::high_resolution_clock::now().time_since_epoch().count());
    return device() ^ time;
}
```

2.4.3 Функция `seed_random`

Функция задаёт начальное состояние генератора. На вход подаётся зерно `seed`, выходных данных нет. Одно и то же зерно даёт одну и ту же последовательность случайных чисел. Текст функции приведён в листинге 15.

Листинг 15 — Функция `seed_random`

```
void seed_random(unsigned seed){
    generator().seed(seed);
}
```

2.4.4 Функция `random_below`

Функция возвращает равновероятное целое число от 0 до `bound` – 1. На вход подаётся `bound` $\geqslant$ 1; при `bound` = 0 верхняя граница `bound - 1` переполняется, поэтому функция с таким значением не вызывается. Число выбирается распределением `std::uniform_int_distribution<size_t>` из генератора `generator()`. Текст функции приведён в листинге 16.

Листинг 16 — Функция `random_below`

```
size_t random_below(size_t bound){
    std::uniform_int_distribution<size_t> distribution(0, bound - 1);
    return distribution(generator());
}
```

2.5 Мультимножество (`Multiset.hpp`, `Multiset.cpp`)

Файлы содержат классы исключений мультимножеств, класс `Multiset` с операциями, функцию автоматического заполнения и функции вывода. Каждая операция выполняется одним проходом по носителю, поэтому её трудоёмкость — $O(2^n)$.

2.5.1 Классы исключений мультимножеств

Класс `MultisetError` — наследник `std::runtime_error` и базовый класс ошибок мультимножеств; его конструктор принимает текст сообщения. От него наследуются четыре класса. Исключение `MultisetIndexError` с сообщением «номер элемента вне носителя» выбрасывается при обращении к кратности элемента с номером $i \geqslant N$. Исключение `CarrierMismatchError` с сообщением «мультимножества построены над разными носителями» выбрасывается при операции над мультимножествами с разными носителями. Исключение `NotSubmultisetError` с сообщением «кратность элемента больше, чем в универсуме» выбрасывается, если аргумент операции не является подмультимножеством универсума. Исключение `CardinalityOverflowError` с сообщением «мощность больше мощности ограничивающего мультимножества» выбрасывается, если для автоматического заполнения задана мощность больше допустимой. Классы приведены в листинге 17.

Листинг 17 — Классы исключений мультимножеств

```
class MultisetError : public std::runtime_error{
public:
    explicit MultisetError(const std::string& message):std::runtime_error(message){}
};

class MultisetIndexError : public MultisetError{
public:
    MultisetIndexError():MultisetError("номер элемента вне носителя"){}
};

class CarrierMismatchError : public MultisetError{
public:
    CarrierMismatchError():MultisetError("мультимножества построены над разными
    носителями"){}
};

class NotSubmultisetError : public MultisetError{
public:
    NotSubmultisetError():MultisetError("кратность элемента больше, чем в универсуме"){}
};

class CardinalityOverflowError : public MultisetError{
public:
    CardinalityOverflowError():MultisetError("мощность больше мощности ограничивающего
    мультимножества"){}
};
```

2.5.2 Класс `Multiset`

Класс реализует мультимножество по определению (1): объект хранит значения функции кратности на всех элементах носителя. Поле `carrier` типа `std::shared_ptr<const CodeSet>` указывает на носитель X; все мультимножества программы ссылаются на один объект носителя. Поле `counts` типа `std::vector<size_t>` длины N содержит кратности: `counts[i]` $= k_A(x_i)$, где x_i — i-е слово кода Грея. Универсум U, мультимножества A, B и результаты операций — объекты этого класса. Закрытые методы `check_same_carrier` и `check_within` проверяют аргументы операций. Функция `random_submultiset` объявлена дружественной, так как изменяет поле `counts` напрямую. Объявление класса и свободных функций приведено в листинге 18.

Листинг 18 — Класс `Multiset`

```
class Multiset{
    std::shared_ptr<const CodeSet> carrier;
    std::vector<size_t> counts;

    void check_same_carrier(const Multiset& other) const;
    void check_within(const Multiset& universe) const;

public:
    explicit Multiset(std::shared_ptr<const CodeSet> carrier);

    const CodeSet& get_carrier() const;
    size_t size() const;
    size_t get_count(size_t i) const;
    void set_count(size_t i, size_t count);
    size_t cardinality() const;
    bool is_submultiset_of(const Multiset& other) const;

    Multiset unite(const Multiset& b) const;
    Multiset intersect(const Multiset& b) const;
    Multiset complement(const Multiset& universe) const;
    Multiset difference(const Multiset& b, const Multiset& universe) const;
    Multiset symmetric_difference(const Multiset& b, const Multiset& universe) const;

    Multiset arithmetic_sum(const Multiset& b, const Multiset& universe) const;
    Multiset arithmetic_difference(const Multiset& b) const;
    Multiset arithmetic_product(const Multiset& b, const Multiset& universe) const;
    Multiset arithmetic_division(const Multiset& b) const;

    friend Multiset random_submultiset(const Multiset& bound, size_t cardinality);
};

Multiset random_submultiset(const Multiset& bound, size_t cardinality);

void print_multiset(std::ostream& out, const Multiset& multiset, size_t limit);
std::ostream& operator<<(std::ostream& out, const Multiset& multiset);
```

2.5.3 Конструктор `Multiset`

Конструктор создаёт мультимножество над носителем, указатель `carrier` на который подаётся на вход, с $k_A(x) = 0$ для всех $x \in X$: длина `counts` равна `carrier->size()`, все элементы равны нулю. Текст конструктора приведён в листинге 19.

Листинг 19 — Конструктор `Multiset`

```
Multiset::Multiset(std::shared_ptr<const CodeSet>
carrier):carrier(carrier),counts(carrier->size(), 0){}
```

2.5.4 Метод `Multiset::get_carrier`

Метод не имеет входных данных и возвращает константную ссылку на носитель; используется при выводе слов. Текст метода приведён в листинге 20.

Листинг 20 — Метод `Multiset::get_carrier`

```
const CodeSet& Multiset::get_carrier() const{
    return *carrier;
}
```

2.5.5 Метод `Multiset::size`

Метод не имеет входных данных и возвращает число N элементов носителя, равное длине `counts`. Текст метода приведён в листинге 21.

Листинг 21 — Метод `Multiset::size`

```
size_t Multiset::size() const{
    return counts.size();
}
```

2.5.6 Метод `Multiset::get_count`

Метод возвращает кратность $k_A(x_i)$. На вход подаётся номер i; при $i \geqslant N$ выбрасывается `MultisetIndexError`. Текст метода приведён в листинге 22.

Листинг 22 — Метод `Multiset::get_count`

```
size_t Multiset::get_count(size_t i) const{
    if(i >= counts.size()){
        throw MultisetIndexError();
    }
    return counts[i];
}
```

2.5.7 Метод `Multiset::set_count`

Метод записывает кратность $k_A(x_i)$ = `count`. На вход подаются номер i и кратность `count`, выходных данных нет. При $i \geqslant N$ выбрасывается `MultisetIndexError`. Условие $k_A(x_i) \leqslant k_U(x_i)$ метод не проверяет: его обеспечивают вызывающие функции. Текст метода приведён в листинге 23.

Листинг 23 — Метод `Multiset::set_count`

```
void Multiset::set_count(size_t i, size_t count){
    if(i >= counts.size()){
        throw MultisetIndexError();
    }
    counts[i] = count;
}
```

2.5.8 Метод `Multiset::cardinality`

Метод не имеет входных данных и возвращает мощность $|A|$, вычисленную по формуле (2) суммированием `counts` за $O(2^n)$. Текст метода приведён в листинге 24.

Листинг 24 — Метод `Multiset::cardinality`

```
size_t Multiset::cardinality() const{
    size_t res = 0;
    for(size_t i = 0; i < counts.size(); i++){
        res += counts[i];
    }
    return res;
}
```

2.5.9 Метод `Multiset::check_same_carrier`

Закрытый метод проверяет, что мультимножество `other`, которое подаётся на вход, построено над тем же носителем: сравниваются указатели `carrier`. При несовпадении выбрасывается `CarrierMismatchError`. Текст метода приведён в листинге 25.

Листинг 25 — Метод `Multiset::check_same_carrier`

```
void Multiset::check_same_carrier(const Multiset& other) const{
    if(carrier != other.carrier){
        throw CarrierMismatchError();
    }
}
```

2.5.10 Метод `Multiset::is_submultiset_of`

Метод проверяет условие $A \subseteq$ `other` по формуле (3). На вход подаётся мультимножество `other`; на выходе получается `true`, если `counts[i]` $\leqslant$ `other.counts[i]` для всех i, и `false` иначе. Сначала вызывается `check_same_carrier`. Текст метода приведён в листинге 26.

Листинг 26 — Метод `Multiset::is_submultiset_of`

```
bool Multiset::is_submultiset_of(const Multiset& other) const{
    check_same_carrier(other);
    for(size_t i = 0; i < counts.size(); i++){
        if(counts[i] > other.counts[i]) return false;
    }
    return true;
}
```

2.5.11 Метод `Multiset::check_within`

Закрытый метод проверяет условие $A \subseteq U$. На вход подаётся универсум `universe`. Если `is_submultiset_of(universe)` возвращает `false`, выбрасывается `NotSubmultisetError`; при разных носителях `CarrierMismatchError` передаётся из `check_same_carrier`. Текст метода приведён в листинге 27.

Листинг 27 — Метод `Multiset::check_within`

```
void Multiset::check_within(const Multiset& universe) const{
    if(!is_submultiset_of(universe)){
        throw NotSubmultisetError();
    }
}
```

2.5.12 Метод `Multiset::unite`

Метод вычисляет объединение $A \cup B$ по формуле (5). На вход подаётся мультимножество B, на выходе получается новое мультимножество над тем же носителем. После `check_same_carrier` для каждого i записывается `max(counts[i], b.counts[i])`. Текст метода приведён в листинге 28.

Листинг 28 — Метод `Multiset::unite`

```
Multiset Multiset::unite(const Multiset& b) const{
    check_same_carrier(b);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = std::max(counts[i], b.counts[i]);
    }
    return res;
}
```

2.5.13 Метод `Multiset::intersect`

Метод вычисляет пересечение $A \cap B$ по формуле (6). На вход подаётся мультимножество B, на выходе получается новое мультимножество. После `check_same_carrier` для каждого i записывается `min(counts[i], b.counts[i])`. Текст метода приведён в листинге 29.

Листинг 29 — Метод `Multiset::intersect`

```
Multiset Multiset::intersect(const Multiset& b) const{
    check_same_carrier(b);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = std::min(counts[i], b.counts[i]);
    }
    return res;
}
```

2.5.14 Метод `Multiset::complement`

Метод вычисляет дополнение $\overline{A}$ по формуле (7); дополнение $\overline{B}$ по формуле (8) вычисляется вызовом этого метода у объекта B. На вход подаётся универсум U, на выходе получается новое мультимножество. Сначала `check_within(universe)` проверяет $A \subseteq U$, поэтому разность `universe.counts[i] - counts[i]` неотрицательна и беззнаковый тип не переполняется. Текст метода приведён в листинге 30.

Листинг 30 — Метод `Multiset::complement`

```
Multiset Multiset::complement(const Multiset& universe) const{
    check_within(universe);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = universe.counts[i] - counts[i];
    }
    return res;
}
```

2.5.15 Метод `Multiset::difference`

Метод вычисляет разность $A \setminus B$ по формуле (10) в виде (9): возвращает `intersect(b.complement(universe))`. Разность $B \setminus A$ по формуле (11) вычисляется вызовом этого метода у объекта B. На вход подаются мультимножество B и универсум U, на выходе получается новое мультимножество. Условие $B \subseteq U$ проверяет `complement`. Текст метода приведён в листинге 31.

Листинг 31 — Метод `Multiset::difference`

```
Multiset Multiset::difference(const Multiset& b, const Multiset& universe) const{
    return intersect(b.complement(universe));
}
```

2.5.16 Метод `Multiset::symmetric_difference`

Метод вычисляет симметрическую разность $A \triangle B$ по формуле (13) в виде (12): возвращает `difference(b, universe).unite(b.difference(*this, universe))`. На вход подаются мультимножество B и универсум U, на выходе получается новое мультимножество. Условия $B \subseteq U$ и $A \subseteq U$ проверяются в двух вызовах `difference`. Текст метода приведён в листинге 32.

Листинг 32 — Метод `Multiset::symmetric_difference`

```
Multiset Multiset::symmetric_difference(const Multiset& b, const Multiset& universe)
const{
    return difference(b, universe).unite(b.difference(*this, universe));
}
```

2.5.17 Метод `Multiset::arithmetic_sum`

Метод вычисляет арифметическую сумму $A + B$ по формуле (14). На вход подаются мультимножество B и универсум U, на выходе получается новое мультимножество. Сначала проверяются условия $A \subseteq U$ и $B \subseteq U$, затем для каждого i записывается `min(counts[i] + b.counts[i], universe.counts[i])`. Так как $k \leqslant 100$, сумма не превосходит 200. Текст метода приведён в листинге 33.

Листинг 33 — Метод `Multiset::arithmetic_sum`

```
Multiset Multiset::arithmetic_sum(const Multiset& b, const Multiset& universe) const{
    check_within(universe);
    b.check_within(universe);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = std::min(counts[i] + b.counts[i], universe.counts[i]);
    }
    return res;
}
```

2.5.18 Метод `Multiset::arithmetic_difference`

Метод вычисляет арифметическую разность $A - B$ по формуле (15); разность $B - A$ по формуле (16) вычисляется вызовом этого метода у объекта B. На вход подаётся мультимножество B, на выходе получается новое мультимножество. Кратности сравниваются до вычитания: при `counts[i] > b.counts[i]` записывается разность, иначе 0, поэтому беззнаковое вычитание не переполняется. Проверка по универсуму не нужна, так как результат не превосходит $k_A(x)$. Текст метода приведён в листинге 34.

Листинг 34 — Метод `Multiset::arithmetic_difference`

```
Multiset Multiset::arithmetic_difference(const Multiset& b) const{
    check_same_carrier(b);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = (counts[i] > b.counts[i]) ? counts[i] - b.counts[i] : 0;
    }
    return res;
}
```

2.5.19 Метод `Multiset::arithmetic_product`

Метод вычисляет арифметическое произведение $A \cdot B$ по формуле (17). На вход подаются мультимножество B и универсум U, на выходе получается новое мультимножество. Проверки те же, что в `arithmetic_sum`, затем для каждого i записывается `min(counts[i] * b.counts[i], universe.counts[i])`; произведение не превосходит $100 \cdot 100 = 10\,000$. Текст метода приведён в листинге 35.

Листинг 35 — Метод `Multiset::arithmetic_product`

```
Multiset Multiset::arithmetic_product(const Multiset& b, const Multiset& universe) const{
    check_within(universe);
    b.check_within(universe);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = std::min(counts[i] * b.counts[i], universe.counts[i]);
    }
    return res;
}
```

2.5.20 Метод `Multiset::arithmetic_division`

Метод вычисляет арифметическое деление A/B по формуле (18); деление B/A по формуле (19) вычисляется вызовом этого метода у объекта B. На вход подаётся мультимножество B, на выходе получается новое мультимножество. При `b.counts[i]` $\neq 0$ записывается целочисленное частное `counts[i] / b.counts[i]`, иначе 0. Текст метода приведён в листинге 36.

Листинг 36 — Метод `Multiset::arithmetic_division`

```
Multiset Multiset::arithmetic_division(const Multiset& b) const{
    check_same_carrier(b);
    Multiset res(carrier);
    for(size_t i = 0; i < counts.size(); i++){
        res.counts[i] = (b.counts[i] != 0) ? counts[i] / b.counts[i] : 0;
    }
    return res;
}
```

2.5.21 Функция `random_submultiset`

Функция выполняет алгоритм автоматического заполнения из подраздела 1.4. На вход подаются ограничивающее мультимножество `bound` (в программе — универсум U) и мощность `cardinality` $= m$; на выходе получается

мультимножество $R \subseteq$ `bound` мощности m. При $m >$ |`bound`| выбрасывается `CardinalityOverflowError`. Вектор `free` реализует список F: он содержит номера элементов, у которых $k_R(x)$ меньше кратности в `bound`. На каждом из m шагов позиция в `free` выбирается вызовом `random_below(free.size())` (п. 2.4.4), кратность элемента увеличивается на 1, а заполненный элемент удаляется из `free` заменой последним за $O(1)$. Трудоёмкость — $O(2^n)$. Текст функции приведён в листинге 37.

Листинг 37 — Функция `random_submultiset`

```
Multiset random_submultiset(const Multiset& bound, size_t cardinality){
    if(cardinality > bound.cardinality()){
        throw CardinalityOverflowError();
    }
    Multiset res(bound.carrier);
    std::vector<size_t> free;
    free.reserve(bound.counts.size());
    for(size_t i = 0; i < bound.counts.size(); i++){
        if(bound.counts[i] > 0) free.push_back(i);
    }
    for(size_t step = 0; step < cardinality; step++){
        size_t j = random_below(free.size());
        size_t i = free[j];
        res.counts[i]++;
        if(res.counts[i] == bound.counts[i]){
            free[j] = free.back();
            free.pop_back();
        }
    }
    return res;
}
```

2.5.22 Функция `print_multiset`

Функция выводит мультимножество в поток в записи из подраздела 1.1, например `{000×2, 011×1}`. На вход подаются поток `out`, мультимножество `multiset` и наибольшее число выводимых пар `limit`; выходных данных нет. Элементы с нулевой кратностью пропускаются. Если элементов с ненулевой кратностью больше `limit`, после `limit` пар выводится «`..., ещё N эл.`», где N — число невыведенных пар. Текст функции приведён в листинге 38.

Листинг 38 — Функция `print_multiset`

```
void print_multiset(std::ostream& out, const Multiset& multiset, size_t limit){
    const CodeSet& carrier = multiset.get_carrier();
    size_t shown = 0;
    size_t hidden = 0;
    out << "{";
    for(size_t i = 0; i < multiset.size(); i++){
        size_t count = multiset.get_count(i);
        if(count == 0) continue;
        if(shown == limit){
            hidden++;
            continue;
        }
        if(shown > 0) out << ", ";
        out << carrier.get_code(i) << "×" << count;
        shown++;
    }
    if(hidden > 0) out << ", ... ещё " << hidden << " эл.";
    out << "}";
}
```

2.5.23 Оператор `operator<<`

Оператор выводит мультимножество в поток полностью: вызывает `print_multiset` с `limit`, равным `size()`, и возвращает поток `out`. На вход подаются поток `out` и мультимножество `multiset`. В программе оператор не вызывается: меню выводит мультимножества через функцию `format_multiset` (п. 2.8.5). Текст оператора приведён в листинге 39.

Листинг 39 — Оператор `operator<<`

```
std::ostream& operator<<(std::ostream& out, const Multiset& multiset){
    print_multiset(out, multiset, multiset.size());
    return out;
}
```

2.6 Универсум (`Universe.hpp`, `Universe.cpp`)

Файлы содержат константу, ограничивающую кратность универсума, класс исключения и функцию построения универсума.

2.6.1 Константа `MAX_MULTIPLICITY` и класс `InvalidMultiplicityError`

Константа `MAX_MULTIPLICITY` = 100 — наибольшая кратность k элемента универсума; её выбор обоснован в подразделе 2.11. Исклю-

чение `InvalidMultiplicityError` — наследник `MultisetError` с сообщением «кратность элемента универсума вне диапазона от 1 до MAX_MULTIPLICITY»; оно выбрасывается при кратности вне отрезка $[1, 100]$. Объявления приведены в листинге 40.

Листинг 40 — Константа `MAX_MULTIPLICITY` и класс `InvalidMultiplicityError`

```
const size_t MAX_MULTIPLICITY = 100;

class InvalidMultiplicityError : public MultisetError{
public:
    InvalidMultiplicityError():MultisetError("кратность элемента универсума вне диапазона
    от 1 до MAX_MULTIPLICITY"){}
};
```

2.6.2 Функция `uniform_universe`

Функция строит универсум U по формуле (4). На вход подаются указатель на носитель `carrier` и кратность `multiplicity` $= k$; на выходе получается мультимножество с $k_U(x) = k$ для всех $x \in X$. При k вне отрезка $[1, \texttt{MAX_MULTIPLICITY}]$ выбрасывается `InvalidMultiplicityError`. Кратности записываются методом `set_count` за $O(2^n)$. Текст функции приведён в листинге 41.

Листинг 41 — Функция `uniform_universe`

```
Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t multiplicity){
    if(multiplicity < 1 || multiplicity > MAX_MULTIPLICITY){
        throw InvalidMultiplicityError();
    }
    Multiset universe(carrier);
    for(size_t i = 0; i < universe.size(); i++){
        universe.set_count(i, multiplicity);
    }
    return universe;
}
```

2.7 Ввод (`Input.hpp`, `Input.cpp`)

Файлы содержат класс исключения конца ввода и функции чтения строк и чисел с проверкой. Весь ввод программы выполняется через эти функции.

2.7.1 Класс `InputClosedError`

Исключение `InputClosedError` — наследник `std::runtime_error` с сообщением «ввод закончился»; оно выбрасывается, когда поток ввода закончился (конец файла или Ctrl+Z). Класс приведён в листинге 42.

Листинг 42 — Класс `InputClosedError`

```
class InputClosedError : public std::runtime_error{
public:
    InputClosedError():std::runtime_error("ввод закончился"){}
};
```

2.7.2 Функция `stdin_is_console`

Функция не имеет входных данных и возвращает `true`, если стандартный ввод связан с консолью, и `false`, если ввод идёт из файла. Функция имеет две реализации, которые выбираются директивой `#ifdef _WIN32`: в Windows вызывается `_isatty(0)`, в других системах — `isatty(0)`. Функция объявлена как `static`. Текст функции приведён в листинге 43.

Листинг 43 — Функция `stdin_is_console`

```
#ifdef _WIN32
#include <io.h>
static bool stdin_is_console(){ return _isatty(0) != 0; }
#else
#include <unistd.h>
static bool stdin_is_console(){ return isatty(0) != 0; }
#endif
```

2.7.3 Функция `read_text`

Функция выводит приглашение и читает одну строку. На вход подаётся строка приглашения `prompt`; на выходе получается прочитанная строка без пробелов, табуляций и символов `\r` по краям. Если строку прочитать не удалось, выбрасывается `InputClosedError`. При вводе из файла, то есть когда `stdin_is_console()` возвращает `false`, прочитанная строка выводится на экран, поэтому в протоколе работы виден весь ввод. Текст функции приведён в листинге 44.

Листинг 44 — Функция `read_text`

```
std::string read_text(const std::string& prompt){
    std::cout << prompt;
    std::string line;
    if(!std::getline(std::cin, line)){
        throw InputClosedError();
    }
    static const bool echo = !stdin_is_console();
    if(echo) std::cout << line.substr(0, line.find('\r')) << std::endl;
    size_t begin = line.find_first_not_of(" \t\r");
    if(begin == std::string::npos) return "";
    size_t end = line.find_last_not_of(" \t\r");
    return line.substr(begin, end - begin + 1);
}
```

2.7.4 Функция `parse_number`

Функция переводит строку в число. На вход подаются строка `text` и ссылка `value`. Если строка состоит из 1–9 десятичных цифр, функция записывает число в `value` и возвращает `true`; иначе возвращает `false`, и `value` не меняется. Ограничение в 9 цифр исключает переполнение: $999\,999\,999 < 2^{32} - 1$. Текст функции приведён в листинге 45.

Листинг 45 — Функция `parse_number`

```
bool parse_number(const std::string& text, size_t& value){
    if(text.empty() || text.size() > 9) return false;
    size_t res = 0;
    for(size_t i = 0; i < text.size(); i++){
        if(text[i] < '0' || text[i] > '9') return false;
        res = res * 10 + (text[i] - '0');
    }
    value = res;
    return true;
}
```

2.7.5 Функция `is_code`

Функция проверяет, что строка является словом заданной длины. На вход подаются строка `text` и разрядность `depth`; на выходе получается `true`, если длина `text` равна `depth` и строка состоит из символов 0 и 1, и `false` иначе. Текст функции приведён в листинге 46.

Листинг 46 — Функция `is_code`

```
bool is_code(const std::string& text, size_t depth){
    if(text.size() != depth) return false;
    for(size_t i = 0; i < text.size(); i++){
        if(text[i] != '0' && text[i] != '1') return false;
    }
    return true;
}
```

2.7.6 Функция `report_bad_number`

Функция выводит причину, по которой строка не принята как число из отрезка [`min`, `max`]. На вход подаются строка `text` и границы `min`, `max`; выходных данных нет. Различаются три причины: пустая строка, число вне отрезка и строка, не являющаяся целым неотрицательным числом. Функция объявлена как `static`. Текст функции приведён в листинге 47.

Листинг 47 — Функция `report_bad_number`

```
static void report_bad_number(const std::string& text, size_t min, size_t max){
    std::string range = "[" + std::to_string(min) + ".." + std::to_string(max) + "]";
    size_t value = 0;
    if(text.empty()){
        std::cout << "! пустая строка: нужно целое число из " << range << std::endl;
    }
    else if(parse_number(text, value)){
        std::cout << "! " << value << " не входит в " << range << std::endl;
    }
    else{
        std::cout << "! «" << text << "» не целое неотрицательное число; нужно число из "
        << range << std::endl;
    }
}
```

2.7.7 Функция `read_number`

Функция читает число из отрезка [`min`, `max`]. На вход подаются приглашение `prompt` и границы `min`, `max`; на выходе получается введённое число. Запрос повторяется, пока строка не будет принята функцией `parse_number` и число не попадёт в отрезок; при каждом отказе вызывается `report_bad_number`. При конце ввода `InputClosedError` передаётся из `read_text`. Текст функции приведён в листинге 48.

Листинг 48 — Функция `read_number`

```
size_t read_number(const std::string& prompt, size_t min, size_t max){
    while(true){
        std::string line = read_text(prompt);
        size_t value = 0;
        if(parse_number(line, value) && value >= min && value <= max){
            return value;
        }
        report_bad_number(line, min, max);
    }
}
```

2.7.8 Функция `read_number_or_empty`

Функция отличается от `read_number` тем, что принимает пустую строку. На вход подаются приглашение `prompt`, границы `min`, `max` и ссылка `value`. При пустой строке функция возвращает `false`, и `value` не меняется; при числе из отрезка записывает его в `value` и возвращает `true`. Текст функции приведён в листинге 49.

Листинг 49 — Функция `read_number_or_empty`

```
bool read_number_or_empty(const std::string& prompt, size_t min, size_t max, size_t&
value){
    while(true){
        std::string line = read_text(prompt);
        if(line.empty()) return false;
        size_t res = 0;
        if(parse_number(line, res) && res >= min && res <= max){
            value = res;
            return true;
        }
        report_bad_number(line, min, max);
    }
}
```

2.7.9 Функция `read_letter`

Функция читает одну букву из заданного набора. На вход подаются приглашение `prompt`, строка допустимых букв `letters` и флаг `allow_empty`; на выходе получается введённая буква, причём прописные латинские буквы приводятся к строчным. Если `allow_empty` равен `true`, пустая строка допустима, и функция возвращает символ `'\0'`. При любом другом вводе выводится сообщение со списком допустимых букв, и запрос повторяется. Текст функции приведён в листинге 50.

Листинг 50 — Функция `read_letter`

```
char read_letter(const std::string& prompt, const std::string& letters, bool allow_empty){
    std::string listed;
    for(size_t i = 0; i < letters.size(); i++){
        listed += (i > 0 ? " " : "") + std::string(1, letters[i]);
    }
    while(true){
        std::string line = read_text(prompt);
        if(line.empty() && allow_empty) return '\0';
        if(line.size() == 1){
            char letter = line[0];
            if(letter >= 'A' && letter <= 'Z') letter = letter - 'A' + 'a';
            if(letters.find(letter) != std::string::npos) return letter;
        }
        if(line.empty()){
            std::cout << "! пустая строка: введите одну из букв " << listed << std::endl;
        }
        else{
            std::cout << "! «" << line << "» не команда; допустимы буквы " << listed <<
            std::endl;
        }
    }
}
```

2.8 Меню (`Menu.hpp`, `Menu.cpp`)

Файлы содержат константы и вспомогательные функции вывода, а также класс `Menu`, который хранит состояние программы и выполняет команды пользователя. Все функции этого подраздела, кроме методов класса, объявлены как `static`.

2.8.1 Константы вывода

Константа `TABLE_LIMIT` = 32 — наибольшее число строк таблицы кратностей. Константа `PRINT_LIMIT` = 16 — наибольшее число выводимых слов кода и пар мультимножества. Константа `LINE_WIDTH` = 100 — ширина строки в символах, по которой выбирается число выводимых пар. Константа `MATRIX_WIDTH` = 112 — ширина матрицы кратностей в символах. Объявления приведены в листинге 51.

Листинг 51 — Константы вывода

```
static const size_t TABLE_LIMIT = 32;
static const size_t PRINT_LIMIT = 16;
static const size_t LINE_WIDTH = 100;
static const size_t MATRIX_WIDTH = 112;
```

2.8.2 Структура `OperationInfo`, массив `OPERATIONS` и строка `COMMANDS`

Структура `OperationInfo` описывает операцию для вывода: поле `title` содержит название, `notation` — обозначение, `formula` — формулу кратности для справки. Массив `OPERATIONS` содержит 13 таких структур в порядке вывода, константа `OPERATION_COUNT = 13` — их число. Строка `COMMANDS` содержит список команд, который выводится перед каждым запросом команды. Объявления приведены в листинге 52.

Листинг 52 — Структура `OperationInfo`, массив `OPERATIONS` и строка `COMMANDS`

```
struct OperationInfo{
    const char* title;
    const char* notation;
    const char* formula;
};

static const OperationInfo OPERATIONS[] = {
    {"объединение", "A ∪ B", "max(kA, kB)"},
    {"пересечение", "A ∩ B", "min(kA, kB)"},
    {"разность", "A \\ B", "min(kA, kU - kB)"},
    {"разность", "B \\ A", "min(kB, kU - kA)"},
    {"симметрическая разность", "A Δ B", "max(k[A\\B], k[B\\A])"},
    {"дополнение", "¬A", "kU - kA"},
    {"дополнение", "¬B", "kU - kB"},
    {"арифметическая сумма", "A + B", "min(kA + kB, kU)"},
    {"арифметическая разность", "A - B", "max(kA - kB, 0)"},
    {"арифметическая разность", "B - A", "max(kB - kA, 0)"},
    {"арифметическое произведение", "A * B", "min(kA * kB, kU)"},
    {"арифметическое деление", "A / B", "kA div kB; 0 при kB = 0"},
    {"арифметическое деление", "B / A", "kB div kA; 0 при kA = 0"},
};
static const size_t OPERATION_COUNT = sizeof(OPERATIONS) / sizeof(OPERATIONS[0]);

static const char* COMMANDS = "u - универсум, a/b - задать A/B, t - таблица, o - операции,
"
                              "m - матрица, h - справка, q - выход";
```

2.8.3 Функция `text_width`

Функция возвращает ширину строки `text` в символах. В кодировке UTF-8 символ занимает от 1 до 4 байт, и все байты, кроме первого, имеют вид `10xxxxxx`, поэтому считаются только байты b, для которых `(b & 0xC0) != 0x80`. Текст функции приведён в листинге 53.

Листинг 53 — Функция `text_width`

```
static size_t text_width(const std::string& text){
    size_t width = 0;
    for(size_t i = 0; i < text.size(); i++){
        if((static_cast<unsigned char>(text[i]) & 0xC0) != 0x80) width++;
    }
    return width;
}
```

2.8.4 Функции `pad_right` и `pad_left`

Функции дополняют строку пробелами до заданной ширины: `pad_right` — справа, `pad_left` — слева. На вход подаются строка `text` и ширина `width`; на выходе получается дополненная строка. Ширина считается функцией `text_width`; строка шире `width` возвращается без изменений. Тексты функций приведены в листинге 54.

Листинг 54 — Функции `pad_right` и `pad_left`

```
static std::string pad_right(const std::string& text, size_t width){
    size_t current = text_width(text);
    return current >= width ? text : text + std::string(width - current, ' ');
}

static std::string pad_left(const std::string& text, size_t width){
    size_t current = text_width(text);
    return current >= width ? text : std::string(width - current, ' ') + text;
}
```

2.8.5 Функция `format_multiset`

Функция возвращает запись мультимножества `multiset`, которое подаётся на вход, для вывода в меню. При $n \leqslant 4$ выводится до `PRINT_LIMIT` пар, то есть все пары, так как $N \leqslant 16$. При $n > 4$ пара занимает до $n + 6$ символов, и число пар равно $\texttt{LINE_WIDTH}/(n + 6)$, но не меньше 4 и не больше 16. Запись строится вызовом `print_multiset` (п. 2.5.22). Текст функции приведён в листинге 55.

Листинг 55 — Функция `format_multiset`

```
static std::string format_multiset(const Multiset& multiset){
    size_t depth = multiset.get_carrier().get_depth();
    size_t limit = PRINT_LIMIT;
    if(depth > 4){
        size_t element_width = depth + 6;
        limit = std::min(PRINT_LIMIT, std::max<size_t>(4, LINE_WIDTH / element_width));
    }
    std::ostringstream out;
    print_multiset(out, multiset, limit);
    return out.str();
}
```

2.8.6 Функция `compact`

Функция убирает пробелы из обозначения операции `notation`, которое подаётся на вход, и возвращает результат, например `A ∪ B` превращается в `A∪B`. Используется для подписей строк матрицы кратностей. Текст функции приведён в листинге 56.

Листинг 56 — Функция `compact`

```
static std::string compact(const std::string& notation){
    std::string res = notation;
    res.erase(std::remove(res.begin(), res.end(), ' '), res.end());
    return res;
}
```

2.8.7 Класс `Menu`

Класс хранит состояние программы. Поле `carrier` указывает на текущий носитель X. Поля `universe`, `a` и `b` — мультимножества U, A и B над этим носителем. Флаги `universe_ready`, `a_ready` и `b_ready` показывают, что соответствующее мультимножество задано. Открытые методы — конструктор и `run`, остальные методы закрытые. Объявление класса приведено в листинге 57.

Листинг 57 — Класс `Menu`

```
class Menu{
    std::shared_ptr<const CodeSet> carrier;
    Multiset universe;
    Multiset a;
    Multiset b;
    bool universe_ready;
    bool a_ready;
    bool b_ready;

    bool is_available(char command) const;

    void create_universe();
    void fill_multiset(Multiset& multiset, bool& ready, const std::string& name);
    bool fill_manual(Multiset& target, size_t cardinality, const std::string& name) const;
    bool parse_manual_line(const std::string& line, const Multiset& target, size_t left,
                           std::vector<size_t>& indices, std::vector<size_t>& counts)
                           const;

    std::vector<Multiset> compute_operations() const;
    void show_help() const;
    void show_table() const;
    void show_operations() const;
    void show_matrix() const;

public:
    Menu();
    void run();
};
```

2.8.8 Конструктор `Menu`

Конструктор не имеет входных данных. Он создаёт носитель разрядности 0 ($N = 0$) и пустые U, A, B над ним; все флаги равны `false`. Текст конструктора приведён в листинге 58.

Листинг 58 — Конструктор `Menu`

```
Menu::Menu():carrier(std::make_shared<CodeSet>(0)),universe(carrier),a(carrier),b(carrier),
             universe_ready(false),a_ready(false),b_ready(false){}
```

2.8.9 Метод `Menu::is_available`

Метод проверяет, можно ли выполнить команду в текущем состоянии. На вход подаётся буква команды `command`; на выходе получается `true`, если команда доступна. Команды `u` и `h` доступны всегда, остальные требуют

`universe_ready`, а команды `o` и `m` — ещё `a_ready` и `b_ready`. Если команда недоступна, выводится, какие данные не заданы и какой командой их задать. Текст метода приведён в листинге 59.

Листинг 59 — Метод `Menu::is_available`

```
bool Menu::is_available(char command) const{
    if(command == 'u' || command == 'h') return true;
    if(!universe_ready){
        std::cout << "! команда " << command << " недоступна: универсум ещё не создан
        (команда u)" << std::endl;
        return false;
    }
    if((command == 'o' || command == 'm') && (!a_ready || !b_ready)){
        std::string missing;
        if(!a_ready) missing = "A (команда a)";
        if(!b_ready) missing += std::string(missing.empty() ? "" : " и ") + "B (команда
        b)";
        std::cout << "! команда " << command << " недоступна: не задано " << missing <<
        std::endl;
        return false;
    }
    return true;
}
```

2.8.10 Метод `Menu::create_universe`

Метод создаёт новый носитель и универсум; входных и выходных данных нет, значения читаются с клавиатуры. Разрядность n читается вызовом `read_number` из отрезка $[0, \texttt{MAX_DEPTH}]$. При $n = 0$ носитель пуст, и кратность не запрашивается. Иначе k читается вызовом `read_number_or_empty` из отрезка $[1, \texttt{MAX_MULTIPLICITY}]$, при пустой строке $k = 1 + \texttt{random_below(MAX_MULTIPLICITY)}$, и универсум строится вызовом `uniform_universe`. Затем A и B заменяются пустыми мультимножествами над новым носителем, и флаги `a_ready`, `b_ready` сбрасываются. Выводятся первые `PRINT_LIMIT` слов кода, универсум и его мощность, а если A или B были заданы, — сообщение об их сбросе. Текст метода приведён в листинге 60.

Листинг 60 — Метод `Menu::create_universe`

```
void Menu::create_universe(){
    size_t depth = read_number("n [0.." + std::to_string(MAX_DEPTH) + "]: ", 0,
    MAX_DEPTH);
    std::shared_ptr<const CodeSet> new_carrier = std::make_shared<CodeSet>(depth);
    size_t size = new_carrier->size();
    Multiset new_universe(new_carrier);
    size_t multiplicity = 0;
```

```
7
8       if(size == 0){
9           std::cout << "при n = 0 в коде Грея нет слов, кратность не нужна" << std::endl;
10      }
11      else{
12          if(!read_number_or_empty("k [1.." + std::to_string(MAX_MULTIPLICITY) + ", Enter -
            случайно]: ",
13                                   1, MAX_MULTIPLICITY, multiplicity)){
14              multiplicity = 1 + random_below(MAX_MULTIPLICITY);
15              std::cout << "k = " << multiplicity << std::endl;
16          }
17          new_universe = uniform_universe(new_carrier, multiplicity);
18      }
19
20      bool had_multisets = a_ready || b_ready;
21      carrier = new_carrier;
22      universe = new_universe;
23      a = Multiset(carrier);
24      b = Multiset(carrier);
25      universe_ready = true;
26      a_ready = false;
27      b_ready = false;
28
29      if(size > 0){
30          size_t shown = std::min(size, PRINT_LIMIT);
31          size_t per_line = std::max<size_t>(1, LINE_WIDTH / (depth + 1));
32          std::cout << "код Грея, 2^" << depth << " = " << size << ":";
33          for(size_t i = 0; i < shown; i++){
34              if(shown > per_line && i % per_line == 0) std::cout << std::endl;
35              else if(i > 0 || shown <= per_line) std::cout << " ";
36              std::cout << carrier->get_code(i);
37          }
38          std::cout << std::endl;
39          if(shown < size) std::cout << "... и ещё " << size - shown << std::endl;
40      }
41      std::cout << "U = " << format_multiset(universe) << ", |U| = " <<
        universe.cardinality() << std::endl;
42      if(had_multisets) std::cout << "A и B сброшены" << std::endl;
43  }
```

2.8.11 Метод `Menu::parse_manual_line`

Метод разбирает одну строку ручного ввода. На вход подаются строка `line`, заполняемое мультимножество `target`, остаток мощности `left` и пустые векторы `indices` и `counts`, в которые записываются номера элементов и добавляемые кратности. На выходе получается `true`, если строка принята. Строка делится на слова по пробелам; слово имеет вид `код:кратность` или `код`, во втором случае кратность равна 1. Для каждого слова проверяется, что код является словом длины n (`is_code`), кратность — число не меньше 1 (`parse_number`), а кратность не больше свободного места $k_U(x) - k_{\text{target}}(x) - t$, где t — кратность того же x, уже записанная из этой строки. Номер элемента

находит `CodeSet::find` (п. 2.3.11). В конце проверяется, что сумма кратностей строки не превосходит `left`. При первой ошибке выводится её причина и возвращается `false`; `target` метод не меняет. Текст метода приведён в листинге 61.

Листинг 61 — Метод `Menu::parse_manual_line`

```
bool Menu::parse_manual_line(const std::string& line, const Multiset& target, size_t left,
                             std::vector<size_t>& indices, std::vector<size_t>& counts)
                             const{
    size_t depth = carrier->get_depth();
    size_t total = 0;
    std::istringstream words(line);
    std::string word;
    while(words >> word){
        size_t colon = word.find(':');
        std::string code = word.substr(0, colon);
        size_t count = 1;
        if(!is_code(code, depth)){
            std::cout << "! «" << word << "»: код должен состоять из " << depth << "
            символов 0 и 1" << std::endl;
            return false;
        }
        if(colon != std::string::npos && (!parse_number(word.substr(colon + 1), count) ||
        count == 0)){
            std::cout << "! «" << word << "»: после двоеточия нужна кратность - целое
            число от 1" << std::endl;
            return false;
        }
        size_t index = 0;
        carrier->find(code, index);
        size_t taken = target.get_count(index);
        for(size_t i = 0; i < indices.size(); i++){
            if(indices[i] == index) taken += counts[i];
        }
        size_t free = universe.get_count(index) - std::min(taken,
        universe.get_count(index));
        if(count > free){
            std::cout << "! «" << word << "»: у " << code << " свободно " << free << " (kU
            = "
                      << universe.get_count(index) << ", занято " << taken << ")" <<
                      std::endl;
            return false;
        }
        indices.push_back(index);
        counts.push_back(count);
        total += count;
    }
    if(total > left){
        std::cout << "! в строке " << total << " ед., а осталось распределить " << left <<
        std::endl;
        return false;
    }
    return true;
}
```

2.8.12 Метод `Menu::fill_manual`

Метод заполняет мультимножество вручную до заданной мощности. На вход подаются пустое мультимножество `target`, мощность `cardinality` и имя `name` для приглашения. На выходе получается `true`, если заполнение завершено, и `false`, если введена пустая строка. При `cardinality` = 0 метод сразу возвращает `true`. Иначе строки читаются вызовом `read_text`, пока |`target`| < `cardinality`; каждая строка проверяется методом `parse_manual_line`, и если она принята, её кратности прибавляются к `target` методом `set_count`. Текст метода приведён в листинге 62.

Листинг 62 — Метод `Menu::fill_manual`

```
bool Menu::fill_manual(Multiset& target, size_t cardinality, const std::string& name)
const{
    if(cardinality == 0) return true;
    std::cout << "пары код:кратность через пробел (01:2 11), каждого кода не больше "
              << universe.get_count(0) << ", Enter - отмена" << std::endl;
    while(target.cardinality() < cardinality){
        size_t left = cardinality - target.cardinality();
        std::string line = read_text(name + " = " + format_multiset(target) + ", осталось
        " + std::to_string(left) + ": ");
        if(line.empty()) return false;
        std::vector<size_t> indices;
        std::vector<size_t> counts;
        if(parse_manual_line(line, target, left, indices, counts)){
            for(size_t i = 0; i < indices.size(); i++){
                target.set_count(indices[i], target.get_count(indices[i]) + counts[i]);
            }
        }
    }
    return true;
}
```

2.8.13 Метод `Menu::fill_multiset`

Метод задаёт мультимножество A или B. На вход подаются ссылка на мультимножество `multiset`, ссылка на его флаг `ready` и имя `name`; выходных данных нет. Мощность m читается из отрезка $[0, |U|]$, при пустой строке m = `random_below(|U| + 1)`. Способ заполнения читается вызовом `read_letter`: `r` — автоматическое заполнение вызовом `random_submultiset(universe, m)`, `m` — ручное заполнение вызовом `fill_manual`, пустая строка — отмена. Заполнение выполняется в отдельный объект `filled`, и только после успешного завершения он записывается в `multiset`,

а `ready` становится `true`; при отмене `multiset` не меняется. Текст метода приведён в листинге 63.

Листинг 63 — Метод `Menu::fill_multiset`

```
void Menu::fill_multiset(Multiset& multiset, bool& ready, const std::string& name){
    size_t high = universe.cardinality();
    size_t cardinality = 0;
    if(!read_number_or_empty("|" + name + "| [0.." + std::to_string(high) + ", Enter -
    случайно]: ",
                             0, high, cardinality)){
        cardinality = random_below(high + 1);
        std::cout << "|" << name << "| = " << cardinality << std::endl;
    }
    char method = read_letter("r - случайно, m - вручную, Enter - отмена: ", "rm", true);
    if(method == '\0'){
        std::cout << "отменено" << std::endl;
        return;
    }

    Multiset filled(carrier);
    if(method == 'm'){
        if(!fill_manual(filled, cardinality, name)){
            std::cout << "отменено" << std::endl;
            return;
        }
    }
    else{
        filled = random_submultiset(universe, cardinality);
    }

    multiset = filled;
    ready = true;
    std::cout << name << " = " << format_multiset(multiset) << ", |" << name << "| = "
              << multiset.cardinality() << std::endl;
}
```

2.8.14 Метод `Menu::compute_operations`

Метод не имеет входных данных и возвращает вектор из 13 результатов операций над A, B и U в порядке массива `OPERATIONS`. Операции $B \setminus A$, $\overline{B}$, $B - A$ и B / A вычисляются вызовом методов объекта `b` с аргументом `a`. Текст метода приведён в листинге 64.

Листинг 64 — Метод `Menu::compute_operations`

```
std::vector<Multiset> Menu::compute_operations() const{
    std::vector<Multiset> res;
    res.push_back(a.unite(b));
    res.push_back(a.intersect(b));
    res.push_back(a.difference(b, universe));
    res.push_back(b.difference(a, universe));
    res.push_back(a.symmetric_difference(b, universe));
    res.push_back(a.complement(universe));
    res.push_back(b.complement(universe));
    res.push_back(a.arithmetic_sum(b, universe));
    res.push_back(a.arithmetic_difference(b));
    res.push_back(b.arithmetic_difference(a));
    res.push_back(a.arithmetic_product(b, universe));
    res.push_back(a.arithmetic_division(b));
    res.push_back(b.arithmetic_division(a));
    return res;
}
```

2.8.15 Метод `Menu::show_help`

Метод не имеет входных и выходных данных. Он выводит справку: запись мультимножества, обозначение, формулу и название каждой из 13 операций из `OPERATIONS` и допустимые значения n, k, $|A|$, $|B|$. Текст метода приведён в листинге 65.

Листинг 65 — Метод `Menu::show_help`

```
void Menu::show_help() const{
    std::cout << "запись {000×2, 011×1}: код Грея × кратность, {} - пустое; kU, kA, kB -
    кратности в U, A, B" << std::endl;
    for(size_t i = 0; i < OPERATION_COUNT; i++){
        std::cout << pad_right(OPERATIONS[i].notation, 7) <<
        pad_right(OPERATIONS[i].formula, 26)
                  << OPERATIONS[i].title << std::endl;
    }
    std::cout << "0 <= n <= " << MAX_DEPTH << ", 1 <= k <= " << MAX_MULTIPLICITY << ", 0
    <= |A|, |B| <= |U|" << std::endl;
}
```

2.8.16 Метод `Menu::show_table`

Метод не имеет входных и выходных данных. Он выводит таблицу кратностей: для первых `TABLE_LIMIT` слов кода значения $k_U(x)$, $k_A(x)$, $k_B(x)$, а затем строку Σ с мощностями $|U|$, $|A|$, $|B|$ по всем словам. У незаданного мультимножества вместо чисел выводится «-». Ширина столбцов выбирается по числу цифр $|U|$. Текст метода приведён в листинге 66.

Листинг 66 — Метод `Menu::show_table`

```
void Menu::show_table() const{
    size_t code_width = std::max<size_t>(carrier->get_depth(), 3);
    size_t number_width = std::max<size_t>(std::to_string(universe.cardinality()).size(),
    3) + 1;
    std::cout << pad_right("код", code_width) << pad_left("kU", number_width)
              << pad_left("kA", number_width) << pad_left("kB", number_width) <<
              std::endl;
    size_t rows = std::min(universe.size(), TABLE_LIMIT);
    for(size_t i = 0; i < rows; i++){
        std::cout << pad_right(carrier->get_code(i), code_width)
                  << pad_left(std::to_string(universe.get_count(i)), number_width)
                  << pad_left(a_ready ? std::to_string(a.get_count(i)) : "-",
                  number_width)
                  << pad_left(b_ready ? std::to_string(b.get_count(i)) : "-",
                  number_width) << std::endl;
    }
    if(rows < universe.size()){
        std::cout << "... показаны " << rows << " из " << universe.size() << std::endl;
    }
    std::cout << pad_right("Σ", code_width)
              << pad_left(std::to_string(universe.cardinality()), number_width)
              << pad_left(a_ready ? std::to_string(a.cardinality()) : "-", number_width)
              << pad_left(b_ready ? std::to_string(b.cardinality()) : "-", number_width)
              << std::endl;
}
```

2.8.17 Метод `Menu::show_operations`

Метод не имеет входных и выходных данных. Он выводит 13 результатов `compute_operations` с мощностями и проверяет два равенства. Равенство $|\overline{A}| = |U| - |A|$ следует из (7) и (2), равенство $|A \cup B| + |A \cap B| = |A| + |B|$ — из (5), (6) и тождества $\max(a, b) + \min(a, b) = a + b$. Текст метода приведён в листинге 67.

Листинг 67 — Метод `Menu::show_operations`

```
void Menu::show_operations() const{
    std::vector<Multiset> results = compute_operations();
    for(size_t i = 0; i < OPERATION_COUNT; i++){
        std::cout << pad_right(OPERATIONS[i].notation, 6) << "= " <<
        format_multiset(results[i])
                  << ", |" << OPERATIONS[i].notation << "| = " << results[i].cardinality()
                  << std::endl;
    }

    size_t u = universe.cardinality();
    size_t na = a.cardinality();
    size_t nb = b.cardinality();
    size_t complement = results[5].cardinality();
    size_t unite = results[0].cardinality();
```

```
    size_t intersect = results[1].cardinality();
    std::cout << "проверка: |¬A| = |U| - |A|: " << complement << " = " << u << " - " << na
              << (complement == u - na ? " верно" : " НЕВЕРНО") << std::endl;
    std::cout << "проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: " << unite << " + " <<
    intersect << " = " << na << " + " << nb
              << (unite + intersect == na + nb ? " верно" : " НЕВЕРНО") << std::endl;
}
```

2.8.18 Метод `Menu::show_matrix`

Метод не имеет входных и выходных данных. Он выводит матрицу кратностей: строки соответствуют U, A, B и 13 результатам операций, столбцы — словам кода, последний столбец содержит мощности. Ширина столбца равна $\max(n, 3) + 2$ символов, число столбцов — $\min(N, (\texttt{MATRIX_WIDTH} - 13)\ \mathrm{div}(\max(n, 3) + 2))$, где 13 — ширина подписи строки и столбца мощностей. Мощности считаются по всем словам. Текст метода приведён в листинге 68.

Листинг 68 — Метод `Menu::show_matrix`

```
void Menu::show_matrix() const{
    std::vector<Multiset> results = compute_operations();
    std::vector<std::string> titles = {"U", "A", "B"};
    std::vector<const Multiset*> rows = {&universe, &a, &b};
    for(size_t i = 0; i < OPERATION_COUNT; i++){
        titles.push_back(compact(OPERATIONS[i].notation));
        rows.push_back(&results[i]);
    }

    const size_t label_width = 4;
    const size_t total_width = 9;
    size_t column_width = std::max<size_t>(carrier->get_depth(), 3) + 2;
    size_t columns = std::min(universe.size(), (MATRIX_WIDTH - label_width - total_width)
    / column_width);

    std::cout << pad_right("", label_width);
    for(size_t j = 0; j < columns; j++) std::cout << pad_left(carrier->get_code(j),
    column_width);
    std::cout << pad_left("|·|", total_width) << std::endl;
    for(size_t i = 0; i < rows.size(); i++){
        std::cout << pad_right(titles[i], label_width);
        for(size_t j = 0; j < columns; j++){
            std::cout << pad_left(std::to_string(rows[i]->get_count(j)), column_width);
        }
        std::cout << pad_left(std::to_string(rows[i]->cardinality()), total_width) <<
        std::endl;
    }
    if(columns < universe.size()){
        std::cout << "... показаны " << columns << " из " << universe.size() << " кодов,
        мощности по всем" << std::endl;
    }
}
```

2.8.19 Метод `Menu::run`

Метод выполняет основной цикл программы; входных и выходных данных нет. На каждом шаге выводится строка `COMMANDS` и читается команда вызовом `read_letter` из букв `u a b t o m h q`. Команда `q` завершает цикл; для остальных проверяется `is_available`, затем вызывается соответствующий метод. Исключение `InputClosedError` из функций ввода передаётся в `main`. Текст метода приведён в листинге 69.

Листинг 69 — Метод `Menu::run`

```
void Menu::run(){
    while(true){
        std::cout << std::endl << COMMANDS << std::endl;
        char command = read_letter("> ", "uabtomhq", false);
        if(command == 'q') return;
        if(!is_available(command)) continue;
        switch(command){
        case 'u': create_universe(); break;
        case 'a': fill_multiset(a, a_ready, "A"); break;
        case 'b': fill_multiset(b, b_ready, "B"); break;
        case 't': show_table(); break;
        case 'o': show_operations(); break;
        case 'm': show_matrix(); break;
        case 'h': show_help(); break;
        }
    }
}
```

2.9 Главная функция (`main.cpp`)

Функция `main` получает на вход число аргументов `argc` и массив аргументов `argv` и возвращает код завершения. В Windows включается вывод в кодировке UTF-8 вызовом `SetConsoleOutputCP(CP_UTF8)`. Если программа запущена с аргументами `--seed N`, зерно равно N, иначе зерно возвращает `make_seed`; зерно передаётся в `seed_random` и выводится первой строкой. Затем создаётся объект `Menu` и вызывается его метод `run`. Исключение `InputClosedError` перехватывается, выводится «ввод закончился», и программа возвращает 0. Любое другое исключение — наследник `std::exception` — выводится с префиксом «Внутренняя ошибка:», и программа возвращает 1. Текст функции приведён в листинге 70.

Листинг 70 — Функция `main`

```
int main(int argc, char* argv[]){
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    unsigned seed = make_seed();
    if(argc == 3 && std::strcmp(argv[1], "--seed") == 0){
        seed = static_cast<unsigned>(std::strtoul(argv[2], nullptr, 10));
    }
    seed_random(seed);
    std::cout << "seed = " << seed << std::endl;

    try{
        Menu menu;
        menu.run();
    }
    catch(const InputClosedError&){
        std::cout << std::endl << "ввод закончился" << std::endl;
    }
    catch(const std::exception& error){
        std::cout << "Внутренняя ошибка: " << error.what() << std::endl;
        return 1;
    }
    return 0;
}
```

2.10 Замер времени (`bench.cpp`)

Программа `bench` замеряет время самых долгих действий основной программы при разных n; её результаты использованы в подразделе 2.11.

2.10.1 Функция `measure`

Шаблонная функция получает на вход функциональный объект `action` без аргументов, выполняет его три раза и возвращает наименьшее время выполнения в миллисекундах по часам `std::chrono::steady_clock`. Текст функции приведён в листинге 71.

Листинг 71 — Функция `measure`

```
template<class Action>
static double measure(Action action){
    double best = 1e18;
    for(int run = 0; run < 3; run++){
        auto start = std::chrono::steady_clock::now();
        action();
        auto finish = std::chrono::steady_clock::now();
        best = std::min(best, std::chrono::duration<double, std::milli>(finish -
        start).count());
```

```
    }
    return best;
}
```

2.10.2 Функция `main` программы `bench`

Функция не имеет входных данных. Она задаёт зерно 1 и для каждого n от 10 до `MAX_DEPTH` + 2 = 21 замеряет построение носителя, построение универсума с k = `MAX_MULTIPLICITY`, автоматическое заполнение A с мощностью $|U|$ и вычисление 13 операций над мультимножествами мощностей $|U|/2$ и $|U|/3$. Для каждого n выводятся четыре времени и наибольшее из них. Мощности результатов операций суммируются в переменную `checksum`, значение которой проверяется после замера, чтобы компилятор не удалил вычисления при оптимизации. Текст функции приведён в листинге 72.

Листинг 72 — Функция `main` программы `bench`

```
int main(){
    seed_random(1);
    std::cout << " n   код Грея   универсум   заполнение A   13 операций   наибольшее, мс"
    << std::endl;
    for(size_t depth = 10; depth <= MAX_DEPTH + 2; depth++){
        std::shared_ptr<const CodeSet> carrier;
        double t_code = measure([&]{ carrier = std::make_shared<CodeSet>(depth); });

        Multiset universe(carrier);
        double t_universe = measure([&]{ universe = uniform_universe(carrier,
        MAX_MULTIPLICITY); });

        Multiset a(carrier);
        Multiset b(carrier);
        double t_fill = measure([&]{ a = random_submultiset(universe,
        universe.cardinality()); });
        a = random_submultiset(universe, universe.cardinality() / 2);
        b = random_submultiset(universe, universe.cardinality() / 3);

        size_t checksum = 0;
        double t_operations = measure([&]{
            checksum += a.unite(b).cardinality() + a.intersect(b).cardinality()
                      + a.difference(b, universe).cardinality() + b.difference(a,
                      universe).cardinality()
                      + a.symmetric_difference(b, universe).cardinality()
                      + a.complement(universe).cardinality() +
                      b.complement(universe).cardinality()
                      + a.arithmetic_sum(b, universe).cardinality() +
                      a.arithmetic_difference(b).cardinality()
                      + b.arithmetic_difference(a).cardinality() + a.arithmetic_product(b,
                      universe).cardinality()
                      + a.arithmetic_division(b).cardinality() +
                      b.arithmetic_division(a).cardinality();
        });
```

```
27
28          double worst = std::max({t_code, t_universe, t_fill, t_operations});
29          std::cout << std::fixed << std::setprecision(1) << std::setw(2) << depth
30                    << std::setw(11) << t_code << std::setw(12) << t_universe <<
                      std::setw(15) << t_fill
31                    << std::setw(14) << t_operations << std::setw(17) << worst
32                    << (worst < 1000 ? "" : "  > 1 c") << std::endl;
33          if(checksum == 0) std::cout << "";
34      }
35      return 0;
36  }
```

2.11 Ограничения программы

Ограничения заданы константами `MAX_DEPTH` = 19, `MAX_MULTIPLICITY` = 100 и `MAX_TYPE_DEPTH`, а также константами вывода из `Menu.cpp`. Через меню ввести $n > 19$ нельзя, поэтому исключение `InvalidDepthError` при работе с меню не возникает. Числа вводятся не длиннее 9 цифр.

Значение `MAX_DEPTH` выбрано по замеру времени программой `bench` (подраздел 2.10). Она для каждого n от 10 до 21 строит носитель, универсум с наибольшей кратностью 100, заполняет A с мощностью $|U|$ (худший случай заполнения) и выполняет все 13 операций, каждое действие трижды, и берёт наименьшее время. Самое долгое действие в табл. 3 — автоматическое заполнение: оно делает $|U| = 100 \cdot 2^n$ шагов. При $n = 19$ оно занимает около 0,5 с, а при $n = 20$ уже больше 1 с, поэтому выбрано $n \leqslant 19$. Шаг таймера `std::chrono::steady_clock` в этой сборке равен примерно 0,5 мс, поэтому значения меньше 1 мс в таблице измерены грубо.

Таблица 3 — Время действий в худшем случае, мс

n	Код Грея	Универсум	Заполнение A	13 операций	Наибольшее
10	0,0	0,0	0,0	0,0	0,0
11	0,0	0,0	1,0	0,0	1,0
12	0,0	0,0	2,5	0,0	2,5
13	0,0	0,0	5,0	0,0	5,0
14	0,0	0,0	10,0	0,0	10,0
15	0,0	0,0	21,0	1,0	21,0
16	0,0	0,0	44,1	2,0	44,1
17	0,0	0,0	88,5	4,0	88,5
18	0,0	0,0	234,9	7,5	234,9
19	1,3	1,0	532,2	24,0	532,2
20	3,0	3,0	2741,3	48,0	2741,3
21	7,0	7,0	9408,9	105,1	9408,9

Наибольшая кратность 100 выбрана вместе с `MAX_DEPTH`. Автоматическое заполнение делает $m \leqslant |U| = k \cdot 2^n$ шагов, поэтому в худшем случае $m = |U|$ его время растёт пропорционально k; замер в табл. 3 проводился именно для этого случая при $k = 100$. Значит, при $n \leqslant 19$ и $k \leqslant 100$ любое действие занимает меньше 1 с, а при большей кратности, например $k = 200$, заполнение при $n = 19$ заняло бы, по оценке, около 1 с или больше. С вводом чисел этот предел тоже согласован: при $n = 19$ и $k = 100$ мощность универсума равна $100 \cdot 2^{19} = 52\,428\,800 < 999\,999\,999$, поэтому любую допустимую мощность можно ввести числом не длиннее 9 цифр. Произведение кратностей не больше $100 \cdot 100 = 10\,000$, а сумма мощностей при проверке в `show_operations` не больше $2 \cdot 52\,428\,800$, что меньше наибольшего значения 32-разрядного `size_t`, равного $2^{32} - 1 = 4\,294\,967\,295$; поэтому переполнения нет и в 32-разрядной сборке.

2.12 Обработка ошибок и защита ввода

Весь ввод идёт через функцию `read_text`, а числа и буквы принимаются функциями `read_number`, `read_number_or_empty` и `read_letter` только в допустимом виде; при отказе запрос повторяется с указанием причины (подраздел 2.7). Перед выполнением команды метод `is_available` проверяет, заданы ли нужные для неё данные. Строка ручного ввода при первой ошибке отклоняется целиком методом `parse_manual_line`, а заполнение идёт в копию мультимножества, поэтому ошибка или отмена не меняют заданное ранее мультимножество. Конец ввода (конец файла или Ctrl+Z) передаётся исключением `InputClosedError`; функция `main` перехватывает его, выводит «ввод закончился» и завершает программу с кодом 0.

Исключения ядра защищают классы от неправильного использования; меню проверяет ввод заранее, поэтому при работе с ним они не возникают, а если возникнут, `main` выведет «Внутренняя ошибка:» и вернёт код 1.

3 Результаты работы

Программа запускалась с вводом из текстовых файлов и параметром `--seed`, поэтому все запуски воспроизводимы: номер сценария совпадает с зерном. При вводе из файла программа сама печатает прочитанную строку после приглашения, поэтому на рисунках виден весь ввод. Мультимножества выводятся в записи из подраздела 1.1, столбцы `kU`, `kA`, `kB` содержат значения $k_U(x)$, $k_A(x)$, $k_B(x)$. В выводе программы дополнение $\overline{A}$ печатается как `¬A`, а произведение $A \cdot B$ — как `A * B`. Каждый рисунок показывает одно или несколько связанных действий.

3.1 Корректная работа при $n = 3$

Проверяется основной путь работы: справка, создание универсума при $n = 3$ и $k = 3$, автоматическое заполнение A, ручное заполнение B, таблица кратностей, операции и матрица. Сначала выводится справка с формулами операций и допустимыми значениями (рис. 1); формулы совпадают с формулами (5)–(19).

```
seed = 1

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> h
запись {000×2, 011×1}: код Грея × кратность, {} - пустое; kU, kA, kB - кратности в U, A, B
A ∪ B  max(kA, kB)               объединение
A ∩ B  min(kA, kB)               пересечение
A \ B  min(kA, kU - kB)          разность
B \ A  min(kB, kU - kA)          разность
A Δ B  max(k[A\B], k[B\A])       симметрическая разность
¬A     kU - kA                   дополнение
¬B     kU - kB                   дополнение
A + B  min(kA + kB, kU)          арифметическая сумма
A - B  max(kA - kB, 0)           арифметическая разность
B - A  max(kB - kA, 0)           арифметическая разность
A * B  min(kA * kB, kU)          арифметическое произведение
A / B  kA div kB; 0 при kB = 0   арифметическое деление
B / A  kB div kA; 0 при kA = 0   арифметическое деление
0 <= n <= 19, 1 <= k <= 100, 0 <= |A|, |B| <= |U|
```

Рисунок 1 — Зерно, список команд и справка

После ввода $n = 3$ и $k = 3$ программа выводит 8 слов кода Грея и универсум мощности $3 \cdot 8 = 24$ (рис. 2). Порядок слов совпадает с примером выполнения алгоритма 1.2 в [1].

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 3
k [1..100, Enter - случайно]: 3
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×3, 001×3, 011×3, 010×3, 110×3, 111×3, 101×3, 100×3}, |U| = 24
```

Рисунок 2 — Создание универсума при $n = 3$, $k = 3$

На рис. 3 мультимножество A мощности 10 заполнено автоматически: сумма кратностей $1 + 3 + 1 + 1 + 1 + 3 = 10$, ни одна кратность не превосходит $k_U(x) = 3$. Мультимножество B мощности 8 введено вручную двумя строками; код `110` без двоеточия получил кратность 1.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..24, Enter - случайно]: 10
r - случайно, m - вручную, Enter - отмена: r
A = {000×1, 001×3, 011×1, 010×1, 111×1, 100×3}, |A| = 10

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..24, Enter - случайно]: 8
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 3, Enter - отмена
B = {}, осталось 8: 000:2 011:3 110
B = {000×2, 011×3, 110×1}, осталось 2: 101:2
B = {000×2, 011×3, 110×1, 101×2}, |B| = 8
```

Рисунок 3 — Автоматическое заполнение A ($|A| = 10$) и ручное заполнение B ($|B| = 8$)

Таблица кратностей (рис. 4) показывает $k_U(x), k_A(x), k_B(x)$ для каждого слова и мощности 24, 10 и 8 в строке Σ.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
000   3   1   2
001   3   3   0
011   3   1   3
010   3   1   0
110   3   0   1
111   3   1   0
101   3   0   2
100   3   3   0
Σ    24  10   8
```

Рисунок 4 — Таблица кратностей

Результаты всех 13 операций показаны на рис. 5, а матрица кратностей — на рис. 6. Обе проверки, которые выводит программа, выполнены: $14 = 24 - 10$ и $16 + 2 = 10 + 8$. Видно также, что $|A \triangle B| = 15$, а не $|A \cup B| - |A \cap B| = 14$: это следствие формулы (13).

Ручная проверка проведена для слов `000` ($k_U(x) = 3, k_A(x) = 1, k_B(x) = 2$), `001` $(3, 3, 0)$ и `011` $(3, 1, 3)$ по формулам (5)–(19). Результаты приведены в табл. 4; в последнем столбце — значения из столбцов `000`, `001`, `011` матрицы на рис. 6. Все значения совпали.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {000×2, 001×3, 011×3, 010×1, 110×1, 111×1, 101×2, 100×3}, |A ∪ B| = 16
A ∩ B = {000×1, 011×1}, |A ∩ B| = 2
A \ B = {000×1, 001×3, 010×1, 111×1, 100×3}, |A \ B| = 9
B \ A = {000×2, 011×2, 110×1, 101×2}, |B \ A| = 7
A Δ B = {000×2, 001×3, 011×2, 010×1, 110×1, 111×1, 101×2, 100×3}, |A Δ B| = 15
¬A    = {000×2, 011×2, 010×2, 110×3, 111×2, 101×3}, |¬A| = 14
¬B    = {000×1, 001×3, 010×3, 110×2, 111×3, 101×1, 100×3}, |¬B| = 16
A + B = {000×3, 001×3, 011×3, 010×1, 110×1, 111×1, 101×2, 100×3}, |A + B| = 17
A - B = {001×3, 010×1, 111×1, 100×3}, |A - B| = 8
B - A = {000×1, 011×2, 110×1, 101×2}, |B - A| = 6
A * B = {000×2, 011×3}, |A * B| = 5
A / B = {}, |A / B| = 0
B / A = {000×2, 011×3}, |B / A| = 5
проверка: |¬A| = |U| - |A|: 14 = 24 - 10 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 16 + 2 = 10 + 8 верно
```

Рисунок 5 — Результаты операций и проверки

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m
      000  001  011  010  110  111  101  100      |·|
U       3    3    3    3    3    3    3    3       24
A       1    3    1    1    0    1    0    3       10
B       2    0    3    0    1    0    2    0        8
A∪B     2    3    3    1    1    1    2    3       16
A∩B     1    0    1    0    0    0    0    0        2
A\B     1    3    0    1    0    1    0    3        9
B\A     2    0    2    0    1    0    2    0        7
AΔB     2    3    2    1    1    1    2    3       15
¬A      2    0    2    2    3    2    3    0       14
¬B      1    3    0    3    2    3    1    3       16
A+B     3    3    3    1    1    1    2    3       17
A-B     0    3    0    1    0    1    0    3        8
B-A     1    0    2    0    1    0    2    0        6
A*B     2    0    3    0    0    0    0    0        5
A/B     0    0    0    0    0    0    0    0        0
B/A     2    0    3    0    0    0    0    0        5

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 6 — Матрица кратностей и выход

Таблица 4 — Ручная проверка операций для трёх слов

Опер.	**000**	**001**	**011**	**Вывод**
$A \cup B$	$\max(1,2) = 2$	$\max(3,0) = 3$	$\max(1,3) = 3$	2, 3, 3
$A \cap B$	$\min(1,2) = 1$	$\min(3,0) = 0$	$\min(1,3) = 1$	1, 0, 1
$A \setminus B$	$\min(1, 3-2) = 1$	$\min(3, 3-0) = 3$	$\min(1, 3-3) = 0$	1, 3, 0
$B \setminus A$	$\min(2, 3-1) = 2$	$\min(0, 3-3) = 0$	$\min(3, 3-1) = 2$	2, 0, 2
$A \triangle B$	$\max(1,2) = 2$	$\max(3,0) = 3$	$\max(0,2) = 2$	2, 3, 2
$\overline{A}$	$3-1 = 2$	$3-3 = 0$	$3-1 = 2$	2, 0, 2
$\overline{B}$	$3-2 = 1$	$3-0 = 3$	$3-3 = 0$	1, 3, 0
$A + B$	$\min(1+2, 3) = 3$	$\min(3+0, 3) = 3$	$\min(1+3, 3) = 3$	3, 3, 3
$A - B$	$\max(1-2, 0) = 0$	$\max(3-0, 0) = 3$	$\max(1-3, 0) = 0$	0, 3, 0
$B - A$	$\max(2-1, 0) = 1$	$\max(0-3, 0) = 0$	$\max(3-1, 0) = 2$	1, 0, 2
$A \cdot B$	$\min(1 \cdot 2, 3) = 2$	$\min(3 \cdot 0, 3) = 0$	$\min(1 \cdot 3, 3) = 3$	2, 0, 3
A/B	$1 \operatorname{div} 2 = 0$	$k_B(x) = 0 \Rightarrow 0$	$1 \operatorname{div} 3 = 0$	0, 0, 0
B/A	$2 \operatorname{div} 1 = 2$	$0 \operatorname{div} 3 = 0$	$3 \operatorname{div} 1 = 3$	2, 0, 3

3.2 Некорректный ввод команд и параметров

Проверяется реакция на команды, недоступные в текущем состоянии, и на неверный ввод в меню и при вводе чисел. Пока универсум не создан, команды `a` и `o` не выполняются, и программа подсказывает команду `u` (рис. 7).

```
seed = 2

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
! команда a недоступна: универсум ещё не создан (команда u)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: универсум ещё не создан (команда u)
```

Рисунок 7 — Команды до создания универсума

На рис. 8 в меню введены строка из трёх букв, цифра, пустая строка, повторённая буква и отрицательное число. Каждый раз выводится причина и повторяется приглашение. Прописная буква `U` принята как команда `u`.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> abc
! «abc» не команда; допустимы буквы u a b t o m h q
> 7
! «7» не команда; допустимы буквы u a b t o m h q
>
! пустая строка: введите одну из букв u a b t o m h q
> uu
! «uu» не команда; допустимы буквы u a b t o m h q
> -5
! «-5» не команда; допустимы буквы u a b t o m h q
> U
```

Рисунок 8 — Неверный ввод команды меню

При вводе разрядности отклонены значения `-1`, `20`, `abc`, `3x` и пустая строка, при вводе кратности — `0`, `101` и `-5` (рис. 9). Сообщения различают три причины: не число, число вне отрезка и пустая строка.

```
n [0..19]: -1
! «-1» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: 20
! 20 не входит в [0..19]
n [0..19]: abc
! «abc» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: 3x
! «3x» не целое неотрицательное число; нужно число из [0..19]
n [0..19]:
! пустая строка: нужно целое число из [0..19]
n [0..19]: 2
k [1..100, Enter - случайно]: 0
! 0 не входит в [1..100]
k [1..100, Enter - случайно]: 101
! 101 не входит в [1..100]
k [1..100, Enter - случайно]:  -5
! «-5» не целое неотрицательное число; нужно число из [1..100]
k [1..100, Enter - случайно]: 2
код Грея, 2^2 = 4: 00 01 11 10
U = {00×2, 01×2, 11×2, 10×2}, |U| = 8
```

Рисунок 9 — Неверный ввод разрядности и кратности

На рис. 10 после создания универсума команды `o` и `m` всё ещё недоступны, и программа перечисляет, какие мультимножества не заданы. Затем при вводе мощности A отклонены число 9 (больше $|U| = 8$) и буква, при выборе способа заполнения — буква `z`. Пустая строка отменила заполнение, и команда `o` по-прежнему недоступна.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m
! команда m недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 9
! 9 не входит в [0..8]
|A| [0..8, Enter - случайно]: x
! «x» не целое неотрицательное число; нужно число из [0..8]
|A| [0..8, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: z
! «z» не команда; допустимы буквы r m
r - случайно, m - вручную, Enter - отмена:
отменено

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 10 — Недоступные команды, неверная мощность, неверный способ заполнения и отмена

3.3 Ошибки при ручном заполнении

Проверяется разбор строк ручного ввода. Создан универсум при $n = 2$, $k = 2$, и автоматически заполнено B мощности 3 (рис. 11).

```
seed = 3

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 2
k [1..100, Enter - случайно]: 2
код Грея, 2^2 = 4: 00 01 11 10
U = {00×2, 01×2, 11×2, 10×2}, |U| = 8

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: r
B = {00×1, 11×2}, |B| = 3
```

Рисунок 11 — Подготовка: универсум $n = 2$, $k = 2$ и мультимножество B

На рис. 12 заполняется A мощности 5. Отклонены: код неверной длины `0:1`, код с буквой `0a`, нечисловая кратность `00:x`, нулевая кратность `00:0`, кратность больше $k_U(x) = 2$ `00:3`, повтор кода в одной строке `00:2 00` и

строка с суммой 6 при остатке 5. Строка `00:2 01` принята, после неё повторный ввод кода `00` отклонён, так как у него не осталось места. Пустая строка отменила заполнение.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 5
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 5: 0:1
! «0:1»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 5: 0a
! «0a»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 5: 00:x
! «00:x»: после двоеточия нужна кратность - целое число от 1
A = {}, осталось 5: 00:0
! «00:0»: после двоеточия нужна кратность - целое число от 1
A = {}, осталось 5: 00:3
! «00:3»: у 00 свободно 2 (kU = 2, занято 0)
A = {}, осталось 5: 00:2 00
! «00»: у 00 свободно 0 (kU = 2, занято 2)
A = {}, осталось 5: 00:2 01:2 11:2
! в строке 6 ед., а осталось распределить 5
A = {}, осталось 5: 00:2 01
A = {00×2, 01×1}, осталось 2: 00
! «00»: у 00 свободно 0 (kU = 2, занято 2)
A = {00×2, 01×1}, осталось 2:
отменено
```

Рисунок 12 — Ошибки ручного ввода и отмена

После отмены *A* осталось незаданным: в таблице на рис. 13 в столбце `kA` стоят прочерки, а частично введённые значения не сохранились.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
00    2   -   1
01    2   -   0
11    2   -   2
10    2   -   0
Σ     8   -   3
```

Рисунок 13 — Таблица после отмены заполнения *A*

Затем *A* мощности 3 введено одной строкой, а ручной ввод *B* мощности 4 отменён пустой строкой (рис. 14). Таблица в конце рисунка показывает, что прежнее *B* с кратностями $1, 0, 2, 0$ сохранилось.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 3: 11:2 10
A = {11×2, 10×1}, |A| = 3

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..8, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
B = {}, осталось 4:
отменено

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
00    2   0   1
01    2   0   0
11    2   2   2
10    2   1   0
Σ     8   3   3

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 14 — Успешное заполнение A и отмена ввода B с сохранением прежнего значения

3.4 Граничные значения мощности и кратности

Проверяются мощность, равная мощности универсума, нулевая мощность, наименьшая и наибольшая кратность. Создан универсум при $n = 2$ с наименьшей кратностью $k = 1$, A заполнено автоматически с мощностью $|U| = 4$, B — с мощностью 0 (рис. 15). При $|A| = |U|$ автоматическое заполнение даёт $A = U$.

```
seed = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 2
k [1..100, Enter - случайно]: 1
код Грея, 2^2 = 4: 00 01 11 10
U = {00×1, 01×1, 11×1, 10×1}, |U| = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..4, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: r
A = {00×1, 01×1, 11×1, 10×1}, |A| = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..4, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: r
B = {}, |B| = 0
```

Рисунок 15 — Универсум с $k = 1$, $|A| = |U|$ и $|B| = 0$

Результаты операций (рис. 16) соответствуют формулам: $\overline{A}$ пусто, $\overline{B} = U$, $A \cup B = A \setminus B = A - B = U$, пересечение и произведение пусты. Деление A/B даёт пустое мультимножество, так как по (18) при $k_B(x) = 0$ результат равен нулю.

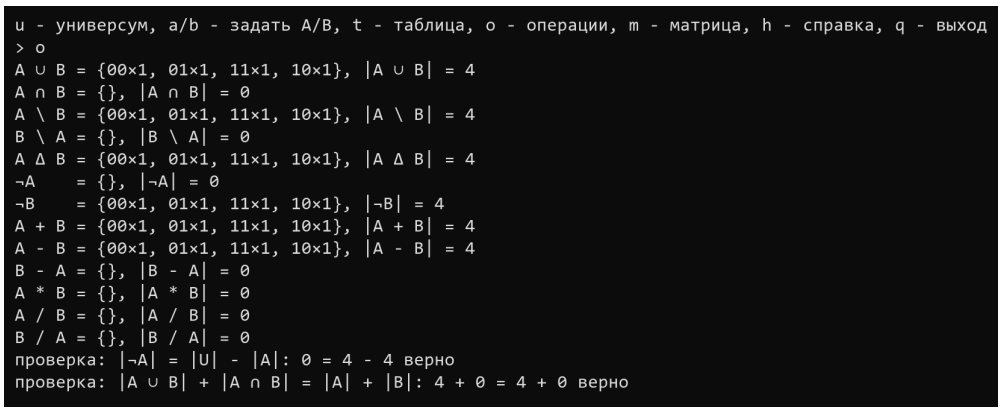


Рисунок 16 — Операции при $A = U$ и $B = \emptyset$

Далее универсум пересоздан при $n = 1$ с наибольшей кратностью $k = 100$, мощность $|U| = 200$; программа сообщила, что A и B сброшены. Мультимножество A мощности 200 введено вручную одной строкой `0:100 1:100`, а при ручном заполнении B с мощностью 0 строки не запрашивались (рис. 17). Операции для этого случая показаны на рис. 18.

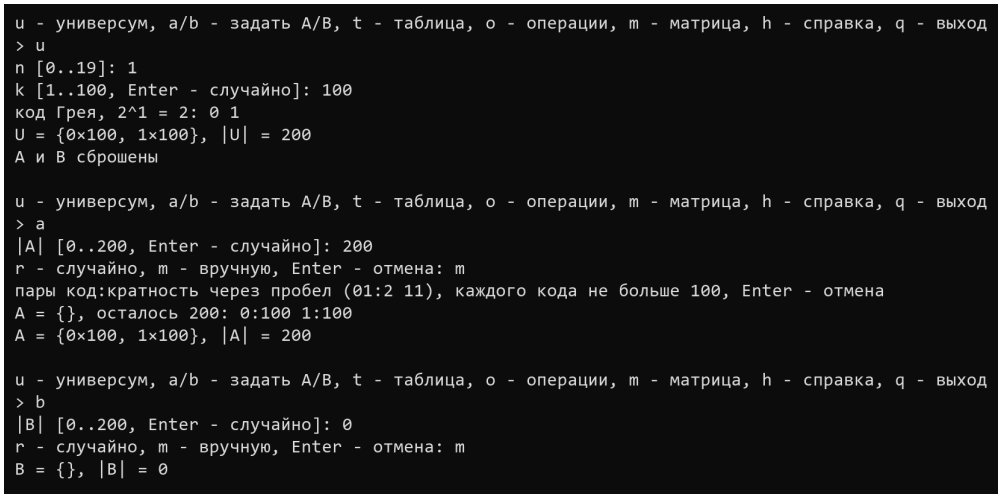


Рисунок 17 — Универсум с $k = 100$ и ручной ввод при $|A| = |U|$ и $|B| = 0$

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {0×100, 1×100}, |A ∪ B| = 200
A ∩ B = {}, |A ∩ B| = 0
A \ B = {0×100, 1×100}, |A \ B| = 200
B \ A = {}, |B \ A| = 0
A Δ B = {0×100, 1×100}, |A Δ B| = 200
¬A    = {}, |¬A| = 0
¬B    = {0×100, 1×100}, |¬B| = 200
A + B = {0×100, 1×100}, |A + B| = 200
A - B = {0×100, 1×100}, |A - B| = 200
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 200 - 200 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 200 + 0 = 200 + 0 верно

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 18 — Операции при $k = 100$

3.5 Пустой универсум, пересоздание и конец ввода

Проверяется разрядность $n = 0$, при которой программа по соглашению из подраздела 1.1 считает код Грея пустым. Кратность в этом случае не запрашивается, универсум пуст, допустимая мощность A и B — только 0, а при пустой строке случайная мощность тоже равна 0 (рис. 19).

```
seed = 5

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 0
при n = 0 в коде Грея нет слов, кратность не нужна
U = {}, |U| = 0

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
Σ     0   -   -

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..0, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: r
A = {}, |A| = 0

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..0, Enter - случайно]:
|B| = 0
r - случайно, m - вручную, Enter - отмена: m
B = {}, |B| = 0
```

Рисунок 19 — Универсум при $n = 0$ и пустые A, B

Все операции дают пустые мультимножества, проверки выполнены (рис. 20). Матрица при $n = 0$ не содержит ни одного столбца с кодом, в ней выводятся только нулевые мощности.

Затем универсум пересоздан при $n = 4$ с пустой строкой вместо кратности: программа выбрала $k = 6$, вывела все 16 слов кода, $|U| = 96$, и сообщила о сбросе A и B (рис. 21). В следующей за этим таблице у A и B стоят прочерки.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {}, |A ∪ B| = 0
A ∩ B = {}, |A ∩ B| = 0
A \ B = {}, |A \ B| = 0
B \ A = {}, |B \ A| = 0
A Δ B = {}, |A Δ B| = 0
¬A    = {}, |¬A| = 0
¬B    = {}, |¬B| = 0
A + B = {}, |A + B| = 0
A - B = {}, |A - B| = 0
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 0 - 0 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 0 + 0 = 0 + 0 верно
```

Рисунок 20 — Операции при $n = 0$

После неё входной файл закончился без команды q, и программа завершилась сообщением «ввод закончился».

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 4
k [1..100, Enter - случайно]:
k = 6
код Грея, 2^4 = 16: 0000 0001 0011 0010 0110 0111 0101 0100 1100 1101 1111 1110 1010 1011 1001 1000
U = {0000×6, 0001×6, 0011×6, 0010×6, 0110×6, 0111×6, 0101×6, 0100×6, 1100×6, 1101×6, 1111×6, 1110×6, 1010×6, 101
1×6, 1001×6, 1000×6}, |U| = 96
A и B сброшены

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код   kU  kA  kB
0000   6   -   -
0001   6   -   -
0011   6   -   -
0010   6   -   -
0110   6   -   -
0111   6   -   -
0101   6   -   -
0100   6   -   -
1100   6   -   -
1101   6   -   -
1111   6   -   -
1110   6   -   -
1010   6   -   -
1011   6   -   -
1001   6   -   -
1000   6   -   -
Σ     96   -   -

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
>
ввод закончился
```

Рисунок 21 — Пересоздание универсума со случайной кратностью, таблица после сброса и конец ввода

3.6 Наибольшая разрядность

Проверяется работа при $n = 19$ и ограничения вывода. Код Грея содержит $2^{19} = 524\,288$ слов; программа выводит первые 16 и число остальных, мощность универсума при $k = 5$ равна 2 621 440 (рис. 22).

Мощность A выбрана случайно и равна 2 341 150, мощность B задана числом 1 000 000 (рис. 23). В записи мультимножеств показаны только пер-

```
seed = 6

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 19
k [1..100, Enter - случайно]: 5
код Грея, 2^19 = 524288:
0000000000000000000 0000000000000000001 0000000000000000011 0000000000000000010 0000000000000000110
0000000000000000111 0000000000000000101 0000000000000000100 0000000000000001100 0000000000000001101
0000000000000001111 0000000000000001110 0000000000000001010 0000000000000001011 0000000000000001001
0000000000000001000
... и ещё 524272
U = {0000000000000000000×5, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 524284
эл.}, |U| = 2621440
```

Рисунок 22 — Универсум при $n = 19$, $k = 5$

вые элементы и число скрытых.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..2621440, Enter - случайно]:
|A| = 2341150
r - случайно, m - вручную, Enter - отмена: r
A = {0000000000000000000×2, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 522901
эл.}, |A| = 2341150

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..2621440, Enter - случайно]: 1000000
r - случайно, m - вручную, Enter - отмена: r
B = {0000000000000000000×2, 0000000000000000001×2, 0000000000000000011×1, 0000000000000000110×1, ... ещё 448234
эл.}, |B| = 1000000
```

Рисунок 23 — Автоматическое заполнение A и B при $n = 19$

Таблица кратностей ограничена 32 строками, но строка Σ содержит мощности по всем элементам (рис. 24).

Обе проверки на рис. 25 выполнены: $280\,290 = 2\,621\,440 - 2\,341\,150$ и $2\,376\,201 + 964\,949 = 2\,341\,150 + 1\,000\,000$. Матрица (рис. 26) показывает 4 столбца, которые помещаются в ширину 112 символов, а мощности считаются по всем словам. После неё ввод $n = 20$ отклонён, так как это больше `MAX_DEPTH`, и ввод закончился.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код                     kU      kA      kB
0000000000000000000      5       2       2
0000000000000000001      5       5       2
0000000000000000011      5       5       1
0000000000000000010      5       5       0
0000000000000000110      5       4       1
0000000000000000111      5       5       1
0000000000000000101      5       5       4
0000000000000000100      5       4       2
0000000000000001100      5       3       1
0000000000000001101      5       5       1
0000000000000001111      5       5       2
0000000000000001110      5       5       3
0000000000000001010      5       5       4
0000000000000001011      5       4       1
0000000000000001001      5       4       2
0000000000000001000      5       5       1
0000000000000011000      5       4       3
0000000000000011001      5       4       2
0000000000000011011      5       5       3
0000000000000011010      5       5       2
0000000000000011110      5       5       2
0000000000000011111      5       5       0
0000000000000011101      5       5       1
0000000000000011100      5       4       1
0000000000000010100      5       4       4
0000000000000010101      5       3       1
0000000000000010111      5       5       2
0000000000000010110      5       3       3
0000000000000010010      5       3       3
0000000000000010011      5       5       1
0000000000000010001      5       5       0
0000000000000010000      5       5       1
... показаны 32 из 524288
Σ                  2621440 2341150 1000000
```

Рисунок 24 — Таблица кратностей при $n = 19$

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {0000000000000000000×2, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 524
056 эл.}, |A ∪ B| = 2376201
A ∩ B = {0000000000000000000×2, 0000000000000000001×2, 0000000000000000011×1, 0000000000000000110×1, ... ещё 447
079 эл.}, |A ∩ B| = 964949
A \ B = {0000000000000000000×2, 0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, ... ещё 498
578 эл.}, |A \ B| = 1530512
B \ A = {0000000000000000000×2, 0000000000000000110×1, 0000000000000000100×1, 0000000000000001100×1, ... ещё 130
946 эл.}, |B \ A| = 189362
A Δ B = {0000000000000000000×2, 0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, ... ещё 506
874 эл.}, |A Δ B| = 1565563
¬A    = {0000000000000000000×3, 0000000000000000110×1, 0000000000000000100×1, 0000000000000001100×2, ... ещё 153
341 эл.}, |¬A| = 280290
¬B    = {0000000000000000000×3, 0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, ... ещё 499
899 эл.}, |¬B| = 1621440
A + B = {0000000000000000000×4, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 524
056 эл.}, |A + B| = 2530512
A - B = {0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, 0000000000000000110×3, ... ещё 460
829 эл.}, |A - B| = 1376201
B - A = {0000000000000101011×1, 0000000000000100101×3, 0000000000001111000×3, 0000000000001000111×2, ... ещё 230
34 эл.}, |B - A| = 35051
A * B = {0000000000000000000×4, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000110×4, ... ещё 447
079 эл.}, |A * B| = 2141208
A / B = {0000000000000000000×1, 0000000000000000001×2, 0000000000000000011×5, 0000000000000000110×4, ... ещё 425
196 эл.}, |A / B| = 1058008
B / A = {0000000000000000000×1, 0000000000000010100×1, 0000000000000010110×1, 0000000000000010010×1, ... ещё 620
68 эл.}, |B / A| = 73975
проверка: |¬A| = |U| - |A|: 280290 = 2621440 - 2341150 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 2376201 + 964949 = 2341150 + 1000000 верно
```

Рисунок 25 — Операции при $n = 19$

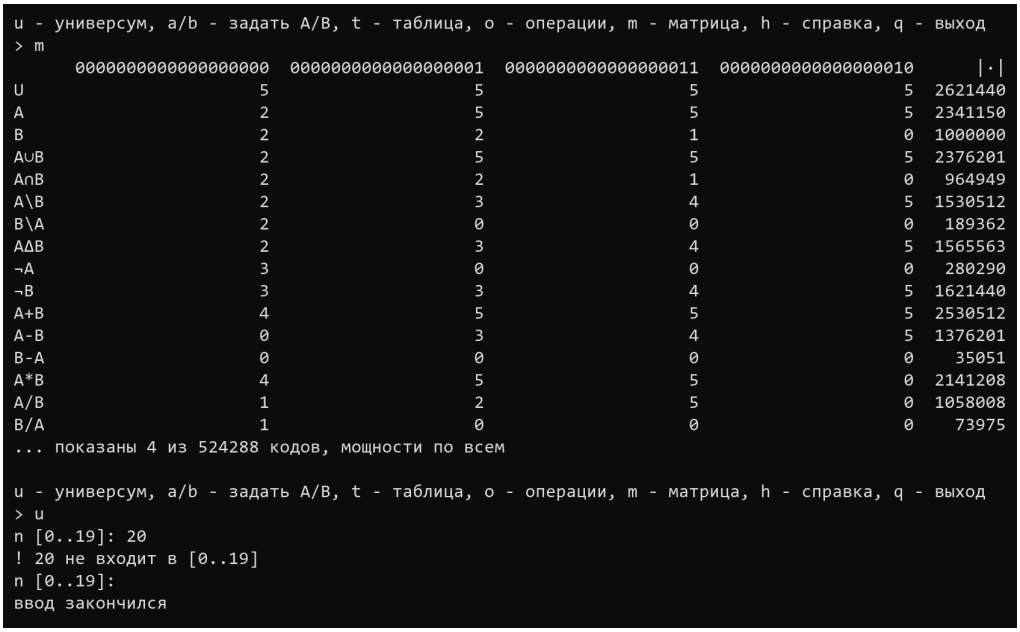


Рисунок 26 — Матрица кратностей при $n = 19$, попытка задать $n = 20$ и конец ввода

ЗАКЛЮЧЕНИЕ

Реализована генерация бинарного кода Грея разрядности n, $0 \leqslant n \leqslant 19$, по алгоритму из [1] с трудоёмкостью $O(2^n)$ и построение по нему универсума U с кратностью $k_U(x) = k$, $1 \leqslant k \leqslant 100$. Реализовано формирование мультимножеств $A, B \subseteq U$ заданной мощности ручным и автоматическим заполнением; трудоёмкость автоматического заполнения $O(2^n)$. Реализованы объединение, пересечение, разности $A \setminus B$ и $B \setminus A$, симметрическая разность, дополнения $\overline{A}$ и $\overline{B}$, арифметические сумма, разности, произведение и деления, трудоёмкость каждой операции $O(2^n)$. Реализованы защита от некорректного ввода с выводом причины отказа и меню, которое не выполняет команду до задания необходимых для неё данных. Работа программы проверена на шести сценариях, включая граничные значения n, k, $|A|$, $|B|$ и обрыв ввода (раздел 3).

Достоинства реализации следующие. Генерация кода Грея выполняется без рекурсии и требует менее $2 \cdot 2^n$ проверок чётности (формула (22)). Номер слова при ручном вводе определяется за $O(n)$ по таблице `positions`, без просмотра кода. По формулам (5)–(19) результат любой операции является подмультимножеством U, поэтому проверка результата не требуется. Разность и симметрическая разность вычисляются через пересечение, объединение и дополнение, без отдельных циклов. Строка ручного ввода с ошибкой отклоняется целиком, а заполнение выполняется в копию мультимножества, поэтому ошибка или отмена не изменяют заданное ранее мультимножество. Параметр `--seed` обеспечивает воспроизводимость запуска.

Недостаток реализации — время автоматического заполнения. На каждую единицу мощности выполняется один случайный выбор, поэтому при $m \leqslant |U| = k \cdot 2^n$ и $k \leqslant 100$ выполняется до $100 \cdot 2^n$ выборов. При $n = 20$ заполнение занимает 2,7 с (табл. 3), из-за этого разрядность ограничена значением 19. Недостаток устраняется выбором кратности каждого элемента за один шаг: элементы $x_1, \ldots, x_N$ обходятся по порядку, и $k_A(x_i)$ выбирается из гипергеометрического распределения по числу ещё не просмотренных единиц U, кратности k и остатку мощности. Число случайных выборов становится равным 2^n независимо от k и m, мощность и условие $A \subseteq U$ сохраняются, интерфейс функции `random_submultiset` не меняется.

Ядро программы не зависит от интерфейса: ядро не выполняет ввод и

вывод, а интерфейс не вычисляет операции. Результат каждой операции является мультимножеством над тем же носителем, поэтому операции можно применять к результатам других операций. Это позволяет без изменения ядра реализовать на основе существующих функций операции над тремя и более мультимножествами, вычисление произвольных выражений из реализованных операций, проверку включения и равенства мультимножеств, построение случайного подмультимножества любого заданного мультимножества, а не только универсума, и проверку тождеств алгебры мультимножеств на случайных данных.

В ходе работы получены навыки представления мультимножества в программе функцией кратности, реализации операций над мультимножествами поэлементными вычислениями над массивом кратностей и обоснования ограничений программы замером времени в худшем случае.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Новиков, Ф. А. Дискретная математика для программистов : учебник для вузов / Ф. А. Новиков. – 3-е изд. – Санкт-Петербург : Питер, 2009. – 384 с. – (Учебник для вузов). – ISBN 978-5-91180-759-7. – Текст : непосредственный.

2. Востров, А. В. Дискретная математика : курс лекций / А. В. Востров ; Санкт-Петербургский политехнический университет Петра Великого. – Санкт-Петербург, 2026. – URL: https://tema.spbstu.ru/dismath_lect/ (дата обращения: 04.10.2026). – Текст : электронный.